Computer Vision 1

HC1b

Camera & Image Formation & Color

Lecturer: Yen-Chia Hsu

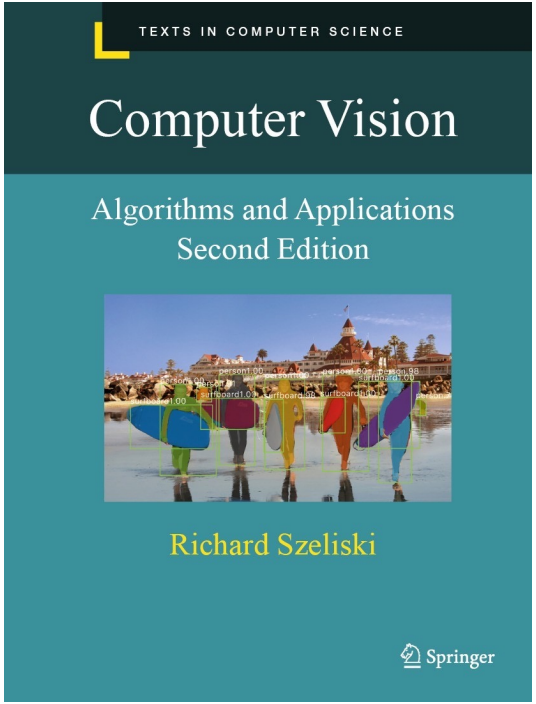
Sep 2026

Credit: This slide is inherited from Dr. Martin Oswald and Dr. Dimitris Tzionas

University of Amsterdam

Textbook – Sections

- 2.1 (and all subsections)
- 2.2 (and all subsections)
- 2.3.1
- 2.3.2



Outline

- Pinhole Camera
- Geometric Image Formation (where an object appears)
- Photometric Image Formation (how bright and what color it is)
- Image Representation
- Color

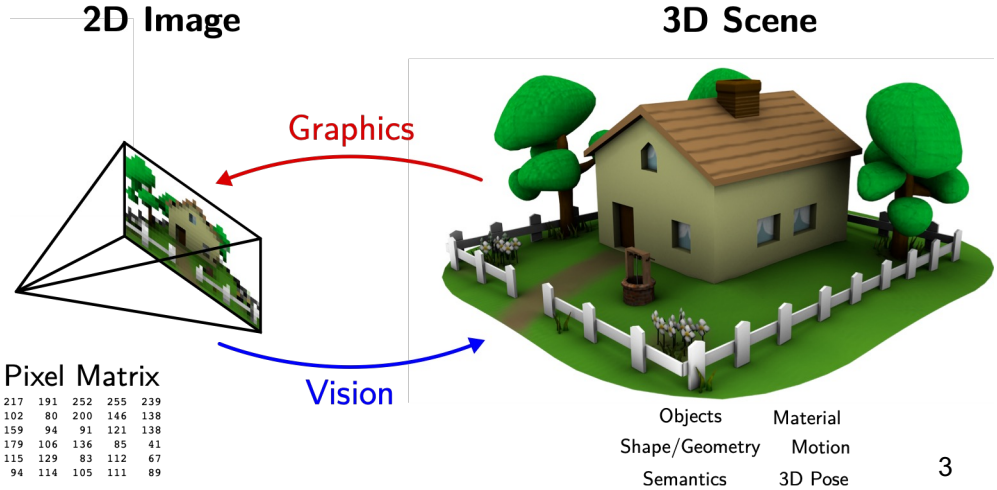


Figure credit:
Andreas Geiger

Human Vision – Retina

Optics of eye → Create a **focused 2D image** of 3D world **on** **retina**

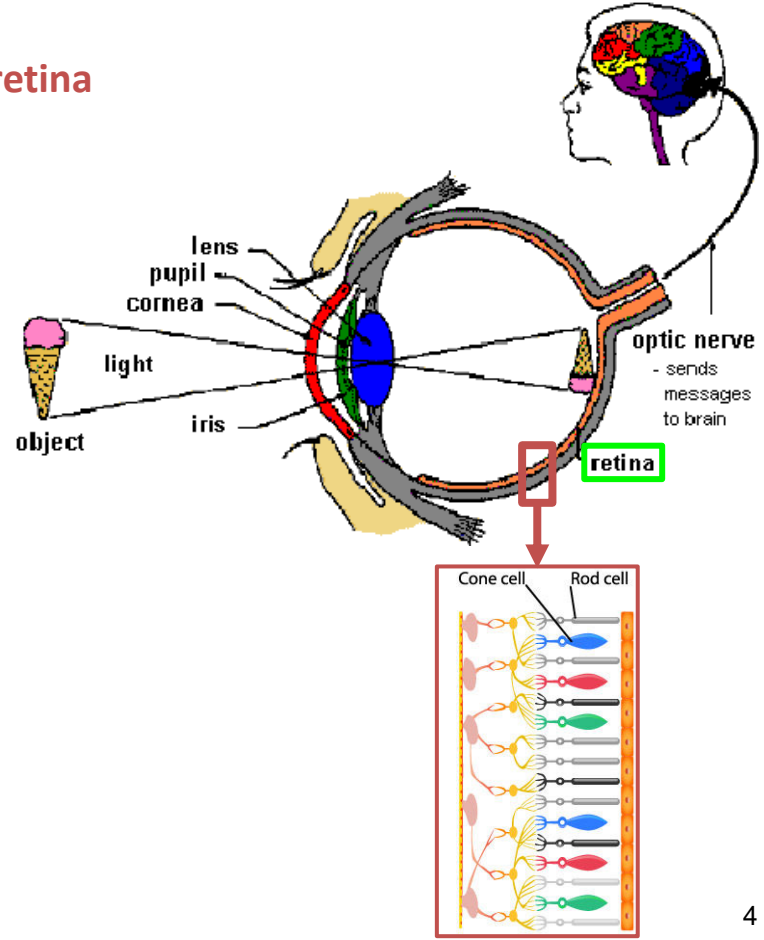
Retina → The eye's innermost & **light-sensitive layer** of tissue

Luminance ~ Sensed in **light-sensitive cells** in **retina**
(visual energy) (**rods** & **cones**)

Collection of measurements → Form 'image'

Collection of pixels on screen → Form digital image

Retina: **Processes** that image &
Send **impulses** along optic nerve to **visual cortex**
to create **visual perception**



Pinhole Camera – Toy Example

Retina @ eye ~~ Image sensor @ camera

Add light-sensitive ‘screen’ in front of an object

Lights reflects onto the object & hits the screen

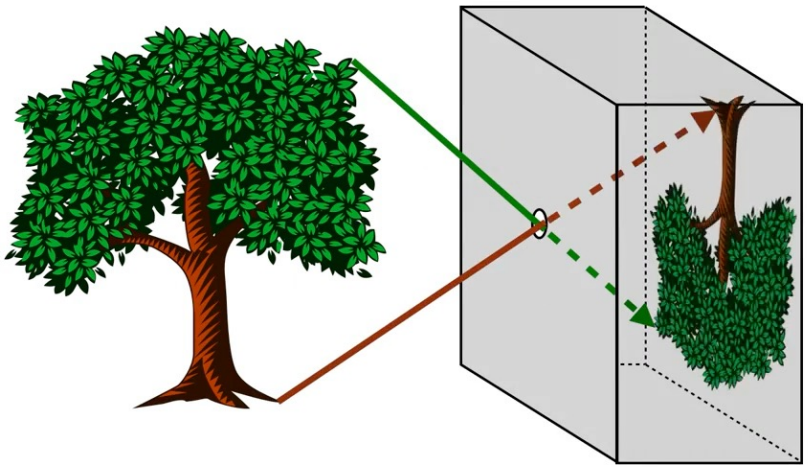
Problem: ∀ object point: Emits light in **all** directions

Image **washed-out** ☹

Sun **washes out++** ☹

Solution: Box with a **hole** → **Filters** *most* light rays

Result: **flipped** image



Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Real Example







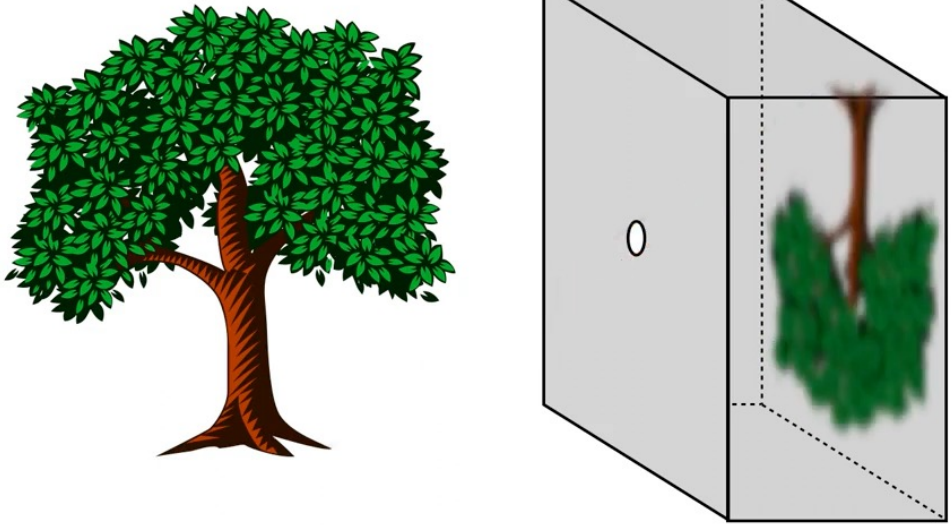
‘Programmatically’

Flip Vertical



Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Aperture



The hole (*aperture*) lets a narrow beam of light through

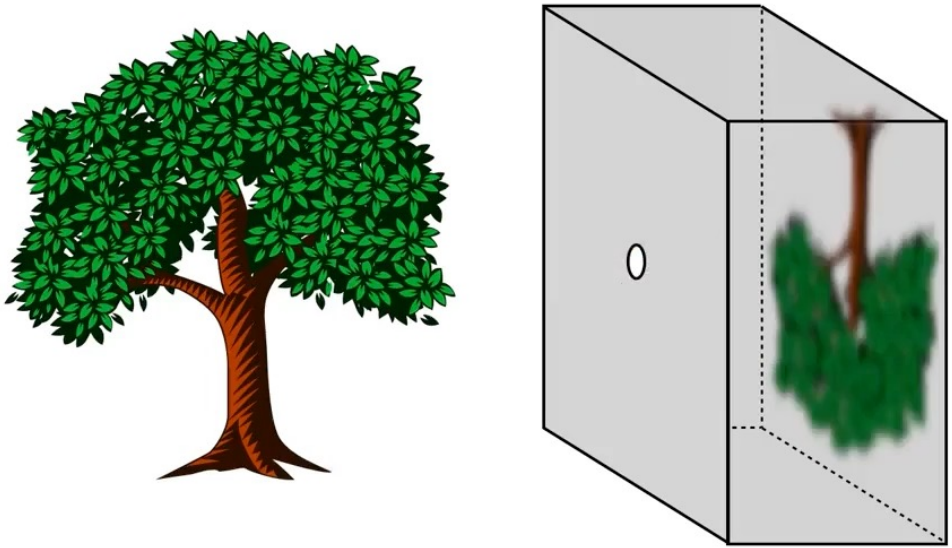
This creates a little *spot of color* on the screen
(that might also *overlap*)

Adds together many overlapping color spots

Result: ***Blurry*** image ☹

Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Aperture Size – Blur

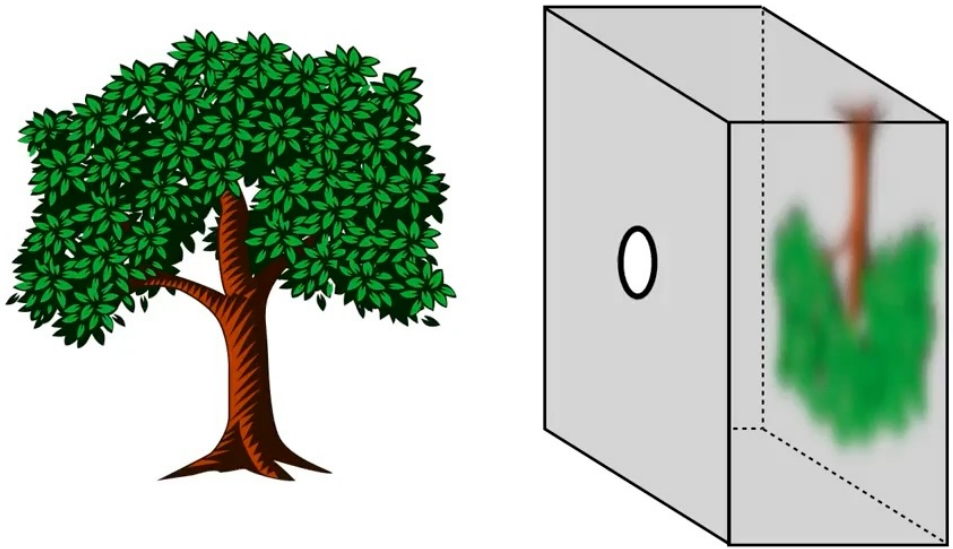


↑ Apperture size
↑ Size of color spots
↑ Blurriness
↓ Sharpness

↓ Apperture size
↓ Size of color spots
↓ Blurriness
↑ Sharpness

Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Aperture Size – Brightness



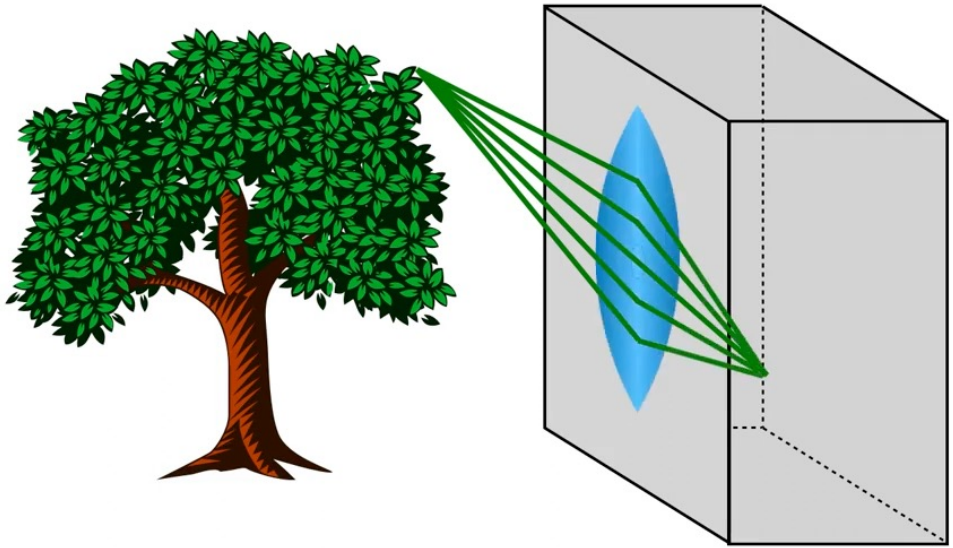
↑ Apperture size
↓ Sharpness
↑ Brightness

Trade off!

↓ Apperture size
↑ Sharpness
↓ Brightness

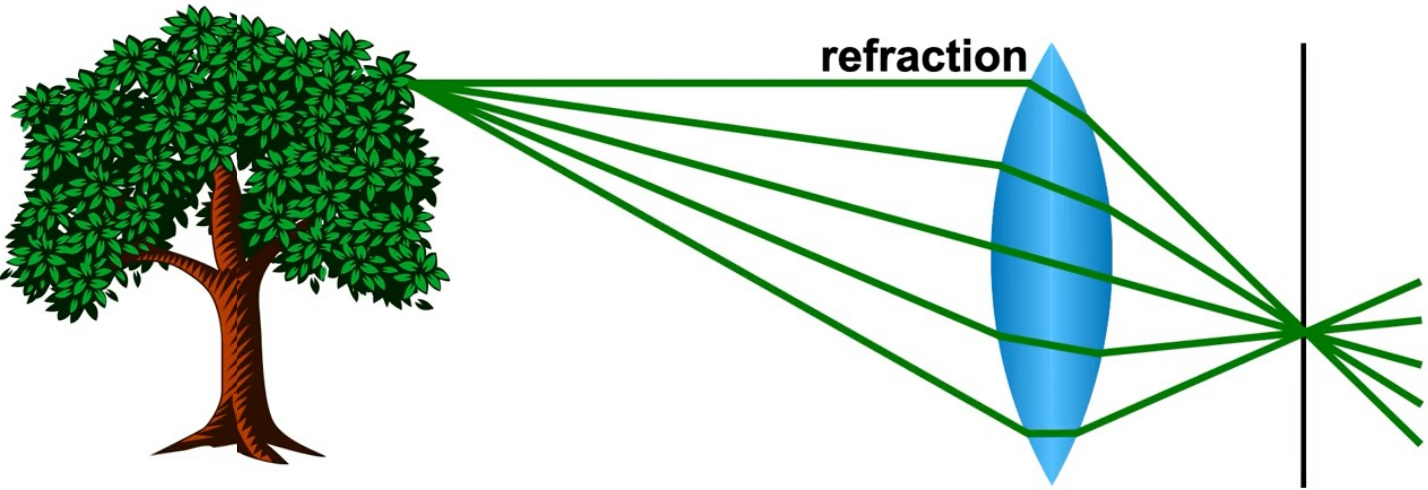
Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Capture more Light



Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

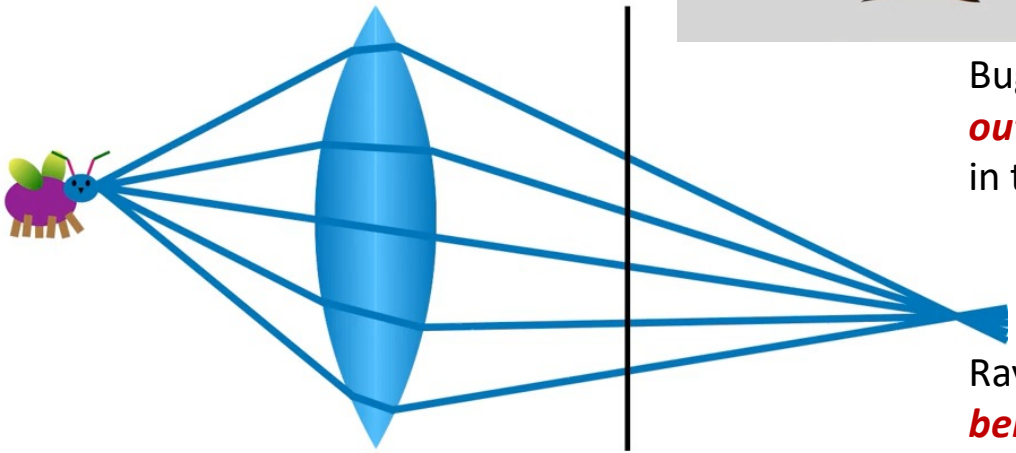
Pinhole Camera – Capture more Light

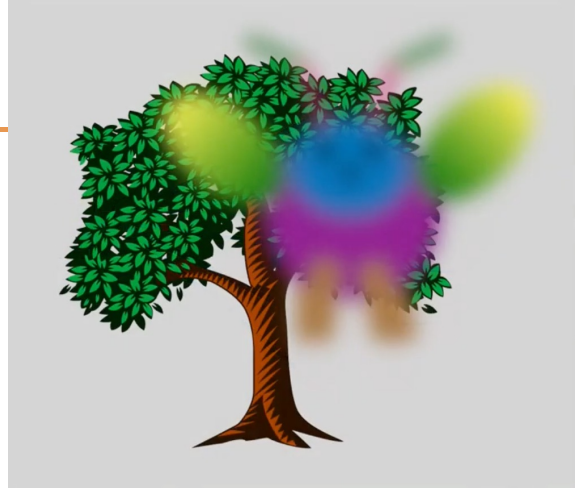


Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Focus







Bug appears ***out of focus*** in the image

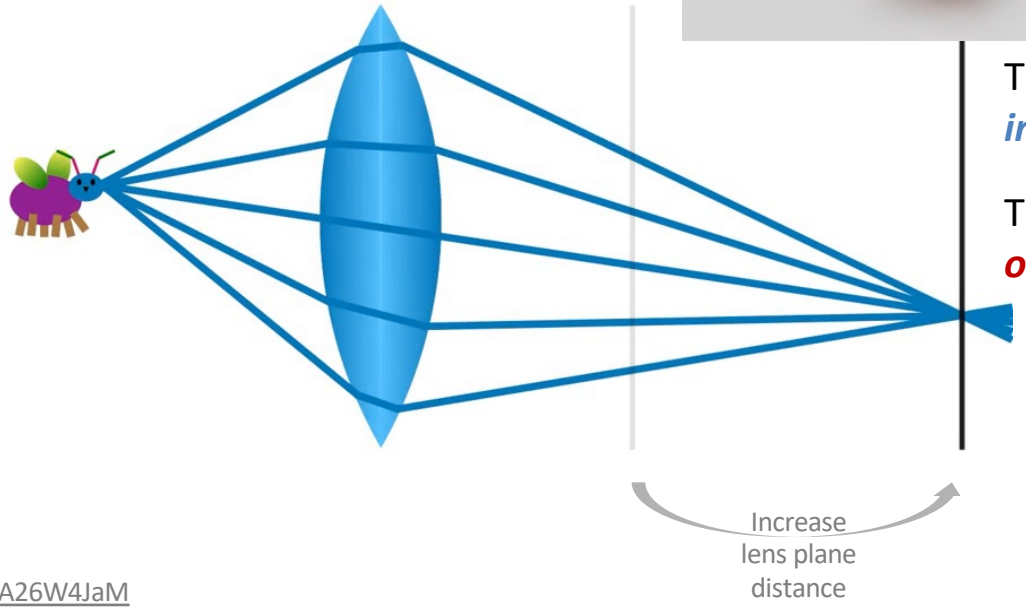
Rays converge ***behind*** the image plane

Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Focus







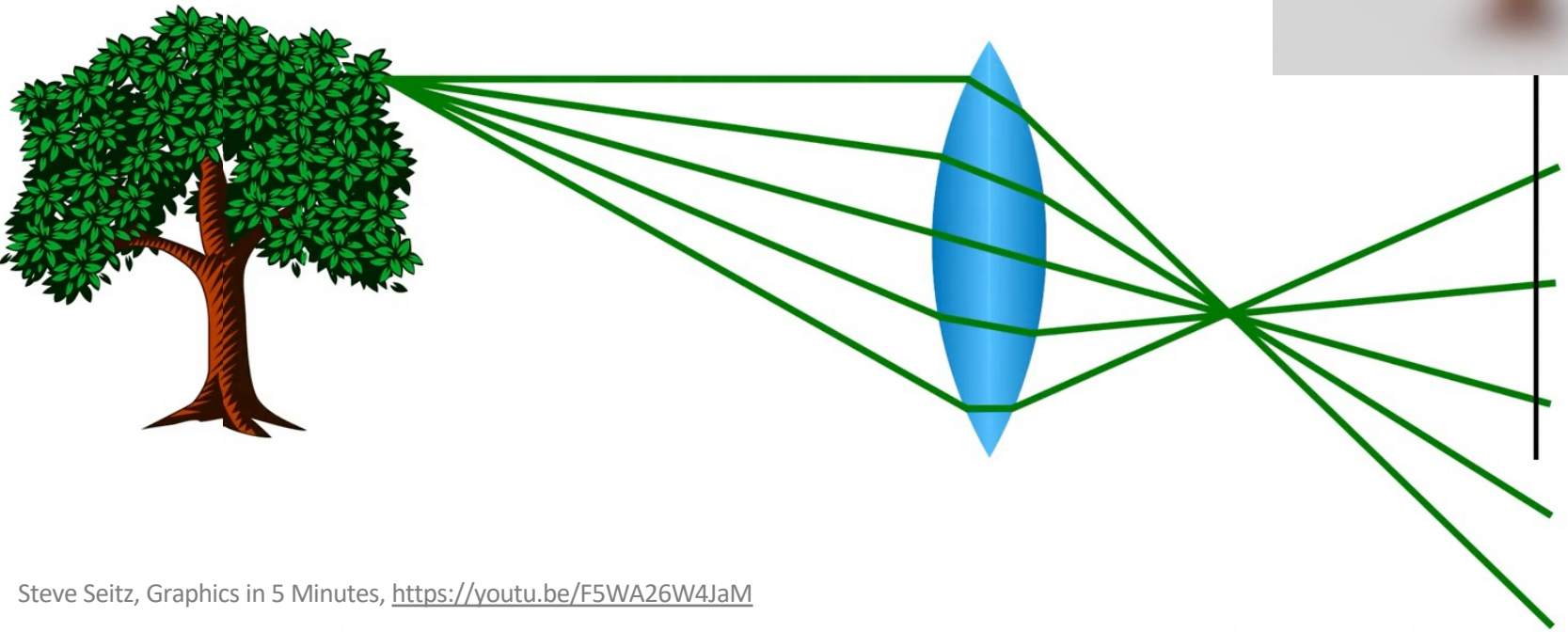
The bug is ***in focus***

The tree is ***out of focus***

Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Focus





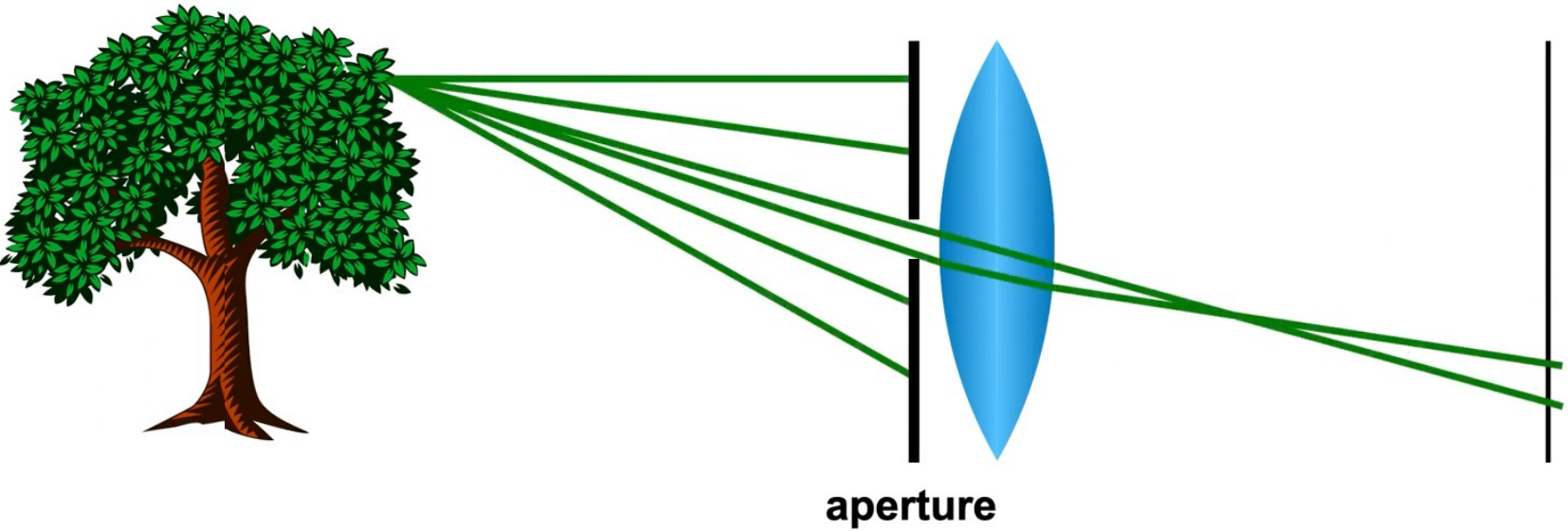
The bug is ***in focus***

The tree is ***out of focus***

as the rays converge far in ***front*** of image plane

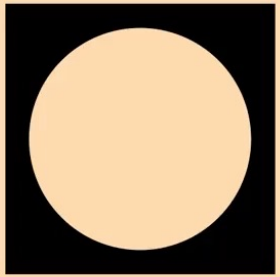
Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Focus

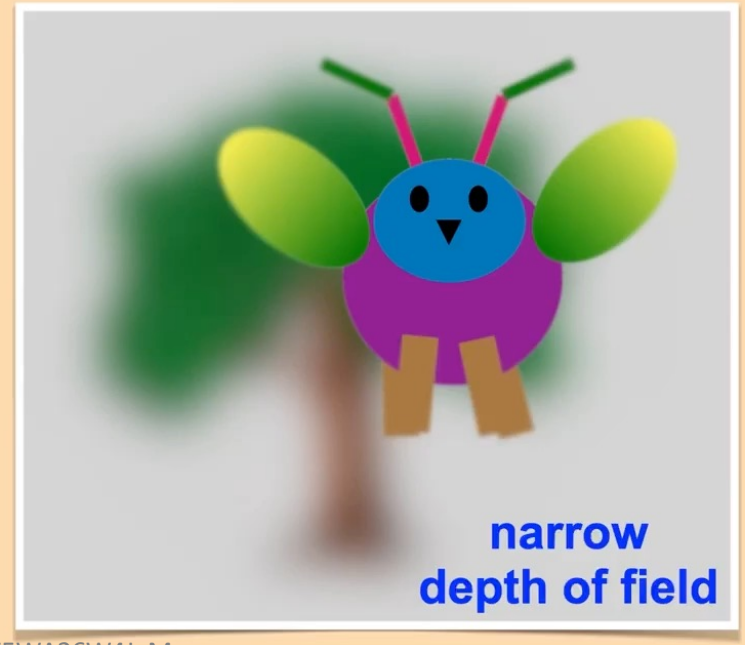


Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Focus



big
aperture

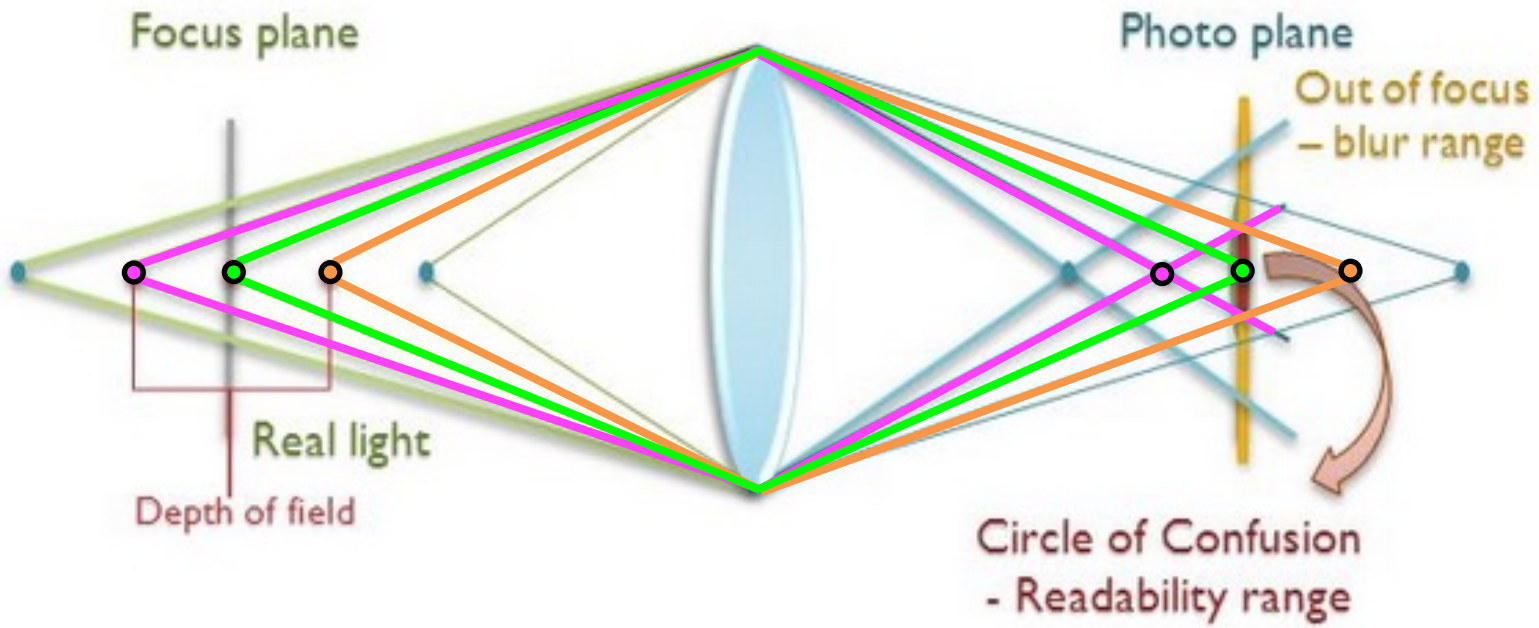


↑ Apperture
↓ Depth of Field

↓ Apperture
↑ Depth of Field

Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Aperture Adjustment – Depth of Field



Whatever lies inside the Depth of Field appears to be in focus (sharp or roughly sharp).

3D World | Inside Camera

Aperture Adjustment – Depth of Field

Disclaimer: The f here is just a dummy symbol. It should not be mistaken with the 'focal length f' defined in later slides



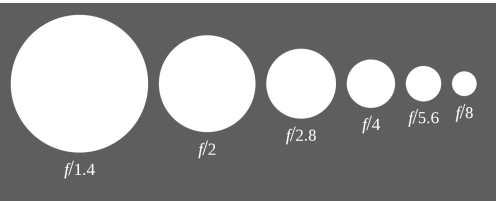


Aperture wide open → f/1.8 f/2.8 f/4.0



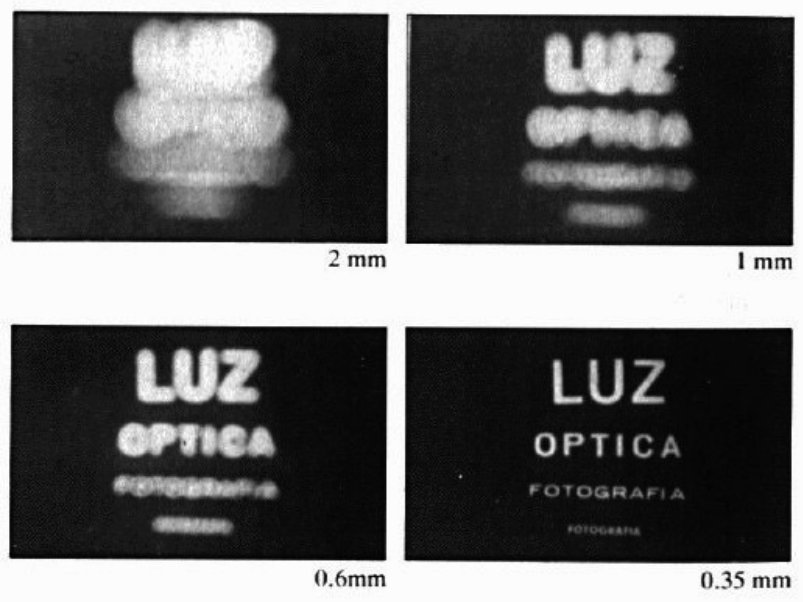


f/5.6 f/16 f/22 ← 'Narrow' Aperture



https://en.wikipedia.org/wiki/F-number

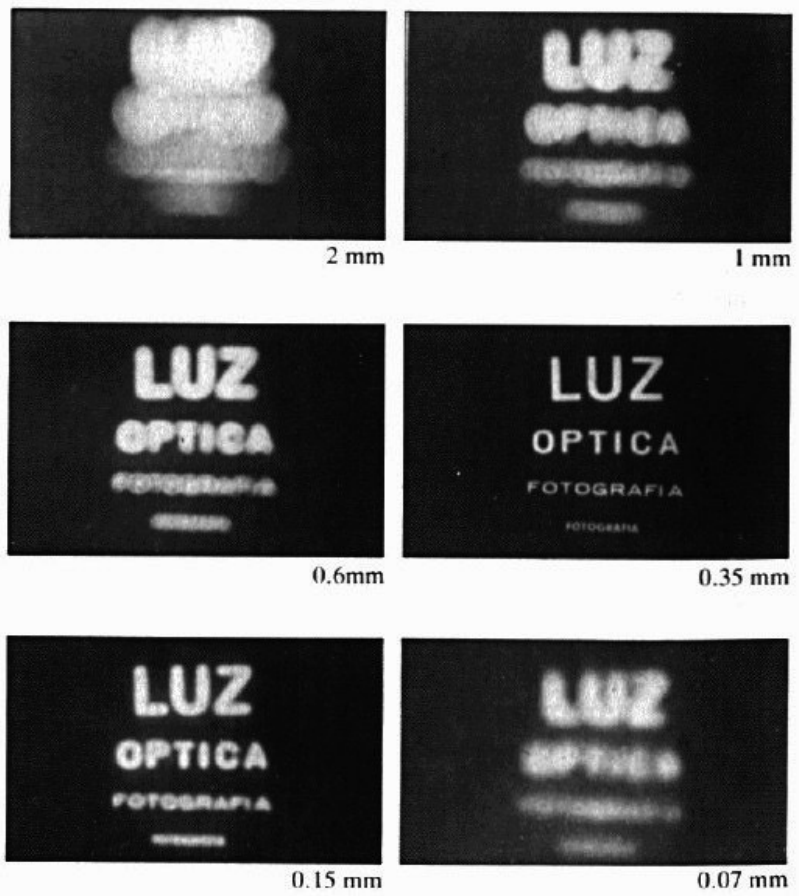
Aperture Shrinking



Why not make the aperture

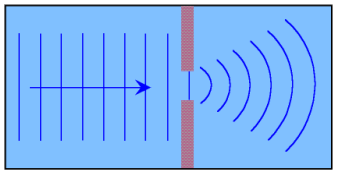
as small as possible?

Aperture Shrinking

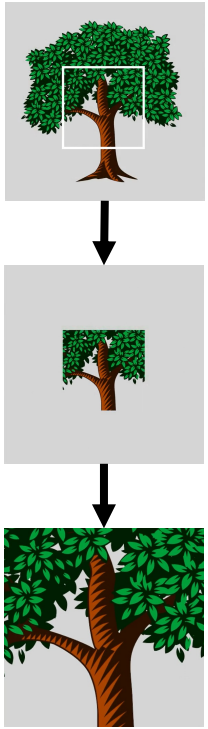


Less light gets through →

Diffraction effects →



Pinhole Camera – Focal Length

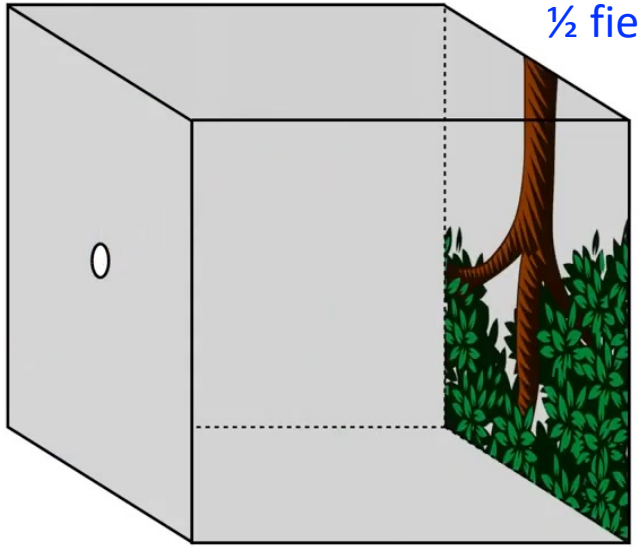




2x magnification

2x magnification

½ field of view



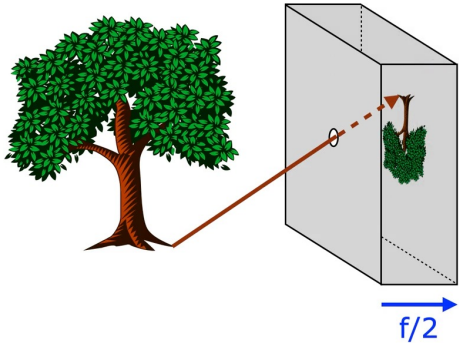
Cannot physically grow camera's body to double f to $2f$

Tele-photo lenses: Image plane far away from optical center

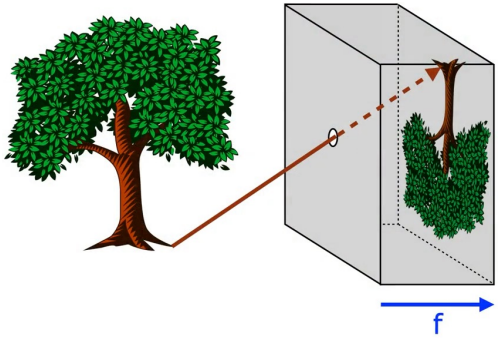


Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Focal Length

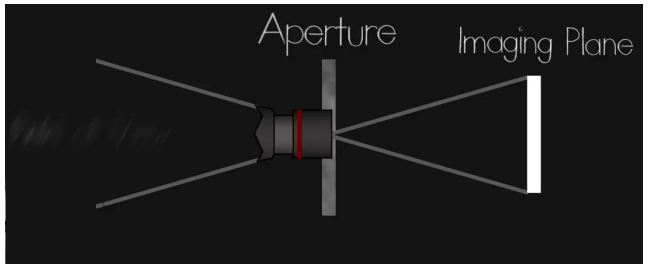


↓ Focal length
↓ Object size (shrinking)
↑ Field of view (more things fit the image)

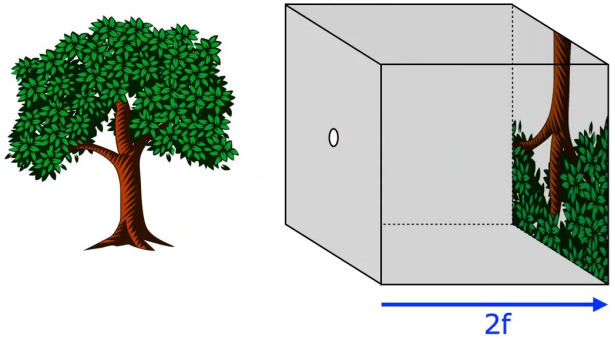


Impossible to move the image plain!

'Moveable' lens & 'bend' light rays & change focus



Mastering the Art of The Dolly Zoom
https://youtu.be/Amj6RiGiTOE



↑ Focal length
↑ Object size (magnification)
↓ Field of view (less things fit the image)

Steve Seitz, Graphics in 5 Minutes
https://youtu.be/F5WA26W4JaM

Outline

- Pinhole Camera
- Geometric Image Formation (where an object appears)
- Photometric Image Formation (how bright and what color it is)
- Image Representation
- Color

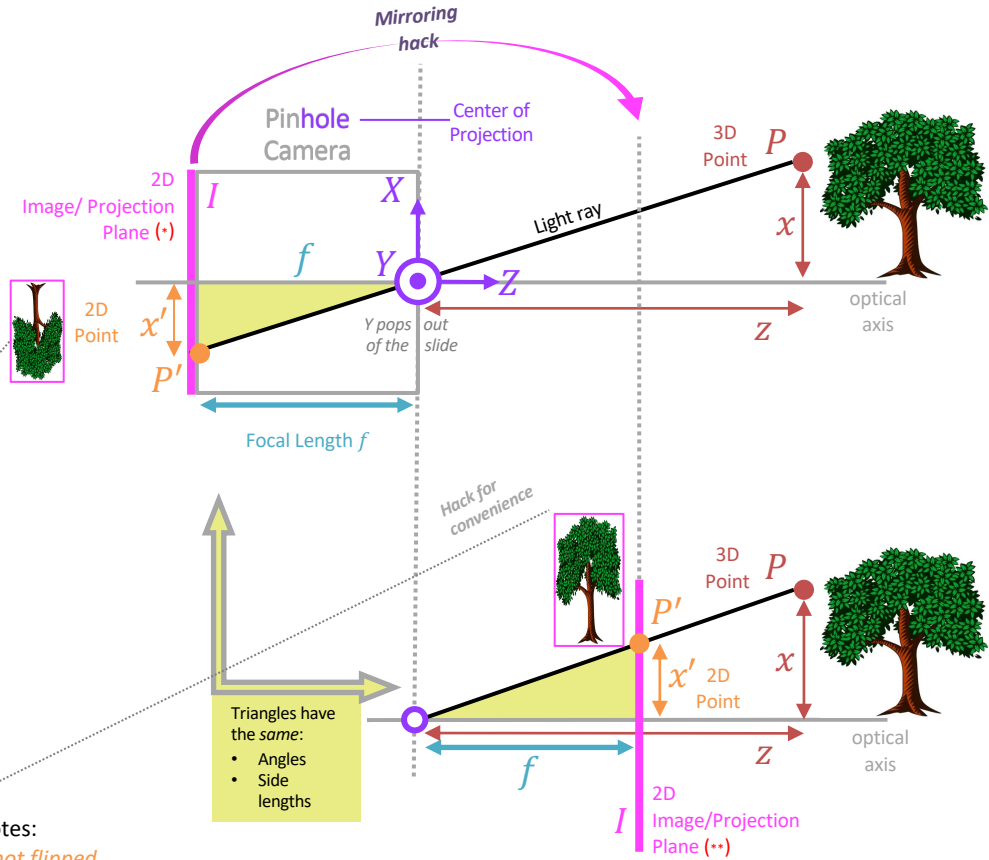
Pinhole Camera – Projection Matrix

Camera Coordinate Frame:

- Origin $\boldsymbol{O}$ at camera center (pinhole)
- Z axis perpendicular to Image Plane I (towards outside the camera)
- X and Y axes *parallel* to Image Plane
- Projection plane is the plane $Z = -f$ (*)
  $Z = +f$ (**)

3D point → Projection → **2D point**

$$P = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \qquad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} -f\frac{x}{z} \\ -f\frac{y}{z} \end{bmatrix} \xrightarrow{\text{mirroring hack}} \begin{bmatrix} f\frac{x}{z} \\ f\frac{y}{z} \end{bmatrix}$$



Minus sign denotes: projected image is *flipped* (horiz. & vertically)

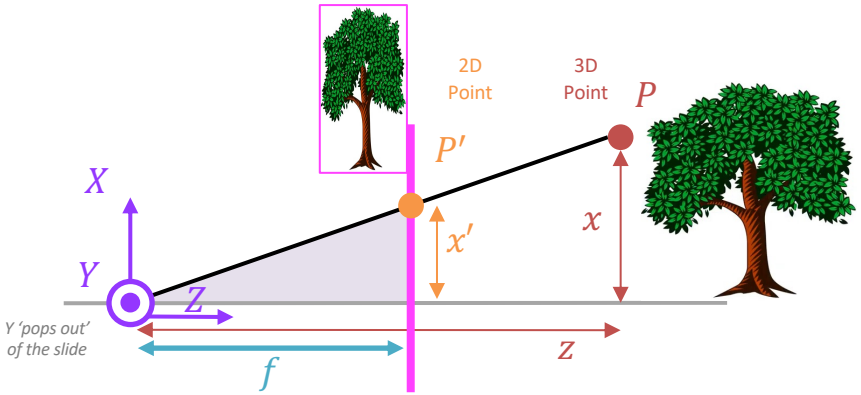
Plus sign denotes: projected image is *not flipped* (see bottom-right plot in this slide)

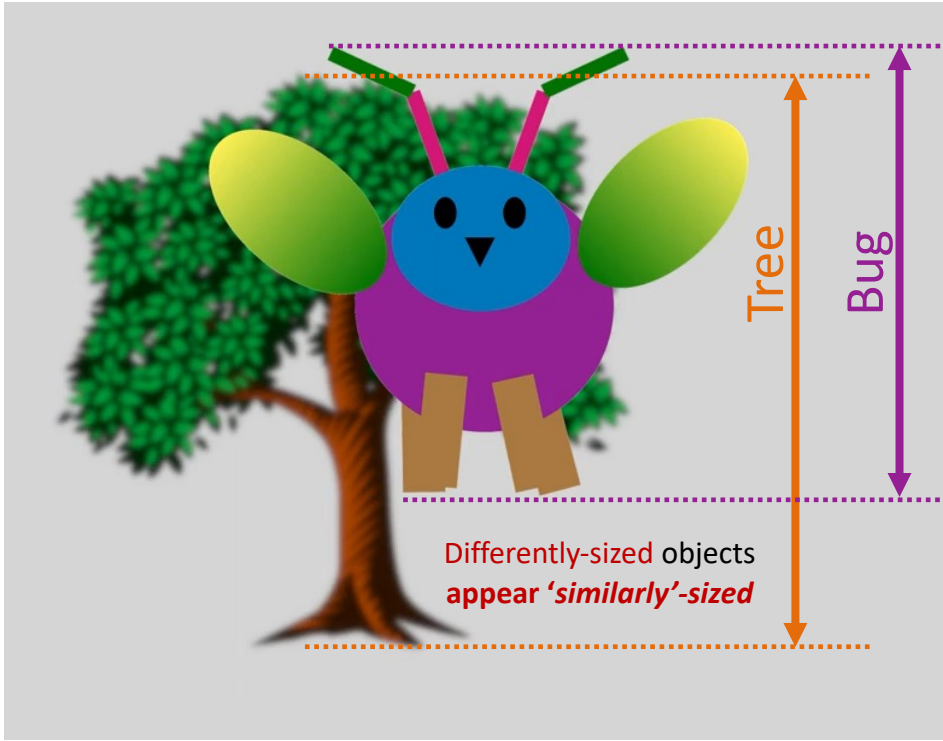
Pinhole Camera – Object Size

Scaled by $\frac{1}{z}$

3D point → Projection → **2D point**

$$P = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \qquad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} f\frac{x}{z} \\ f\frac{y}{z} \end{bmatrix}$$





Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Pinhole Camera – Object Size

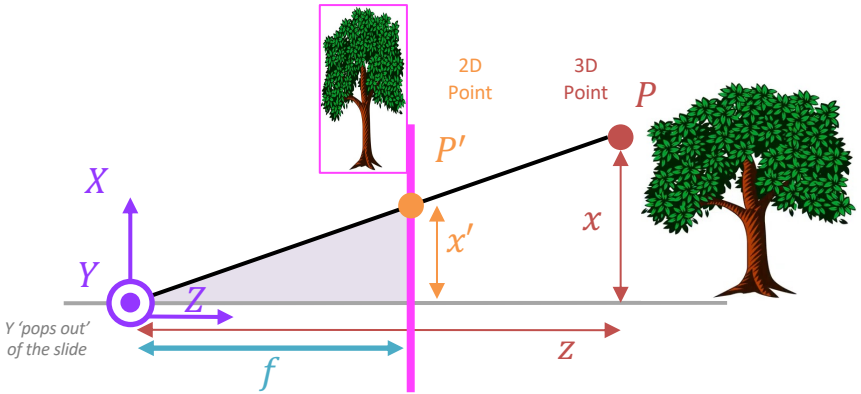
Scaled by $\frac{1}{z}$

3D point → Projection → **2D point**

$$P = \begin{bmatrix} x \\ y \\ z \end{bmatrix} \qquad P' = \begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} f\frac{x}{z} \\ f\frac{y}{z} \end{bmatrix}$$

$$x' = f\frac{x}{z}$$

$$y' = f\frac{y}{z}$$

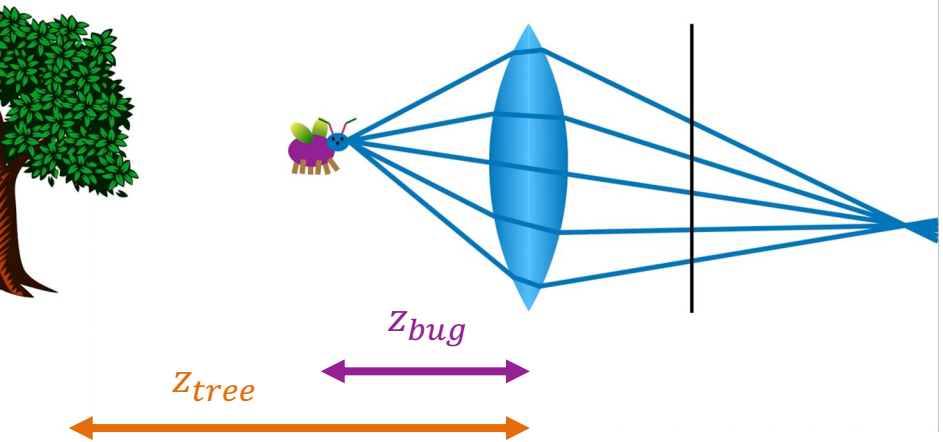


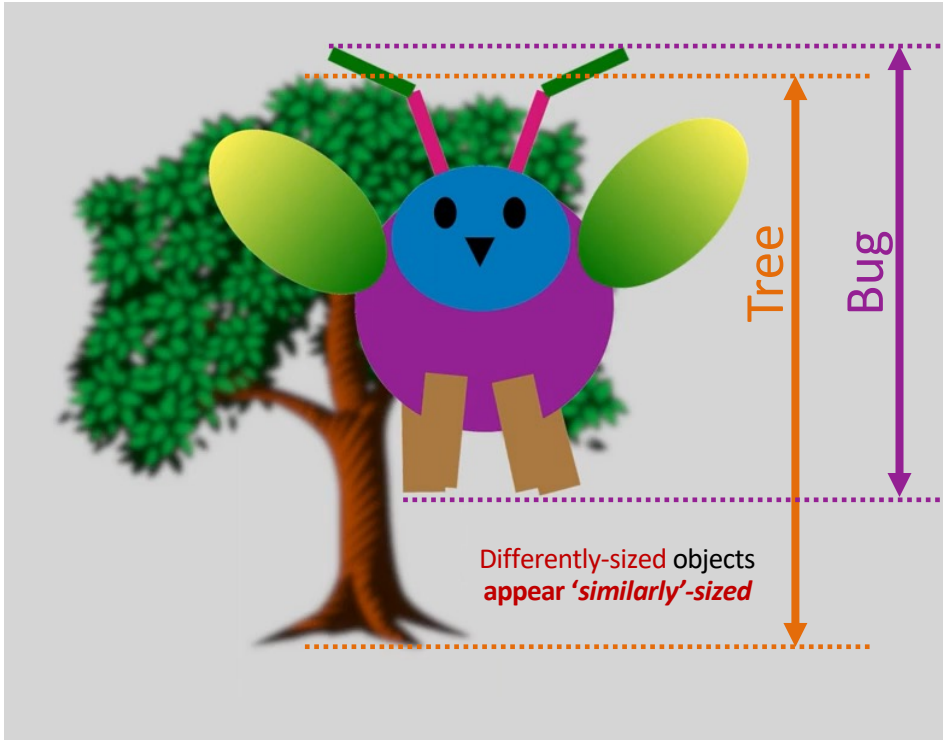
Why?

hint: check the smaller and the larger triangles in the left figure, think about geometry

Pinhole Camera – Object Size

Scaled by $\frac{1}{z}$



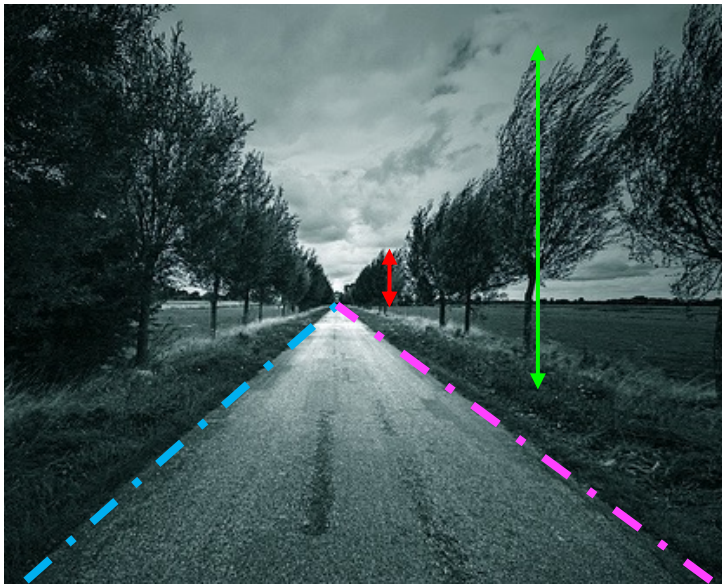


Steve Seitz, Graphics in 5 Minutes, https://youtu.be/F5WA26W4JaM

Projective Geometry

Projective transformation preserves:

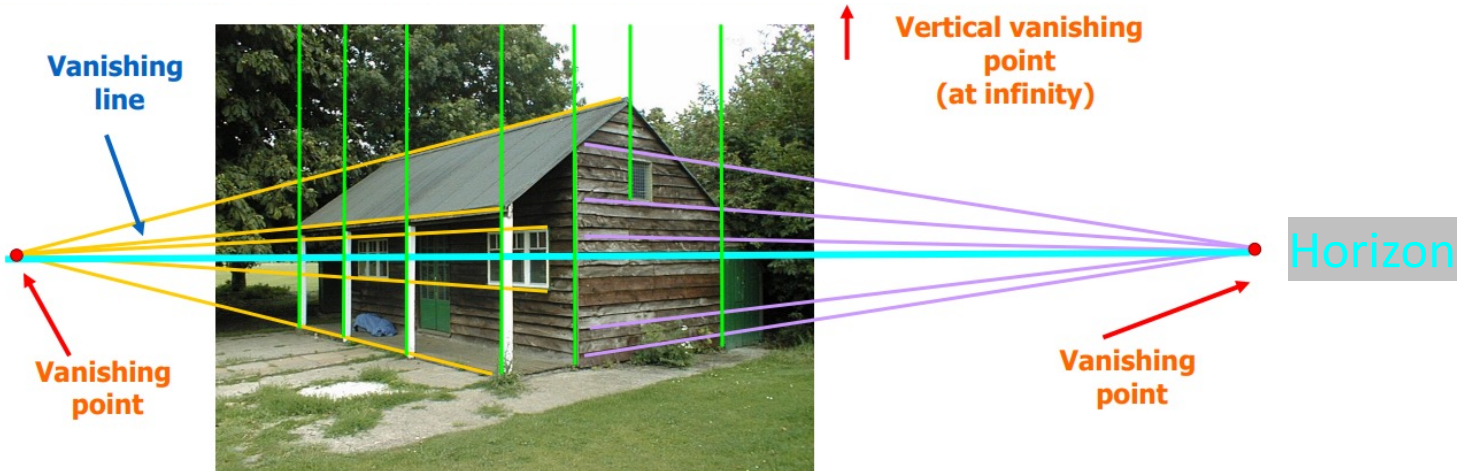
- **Collinearity** of points (straight lines remain straight)
- **Nothing else!** Changes:
  - **Parallelism:** Parallel lines in 3D, after projection **meet** @ a **vanishing point** in 2D
  - **Length:** Same-height Trees: far-away *VS* nearby
    appear: (small) (big)
  - **Angles:** Depend on viewpoint



Vanishing Points & Lines

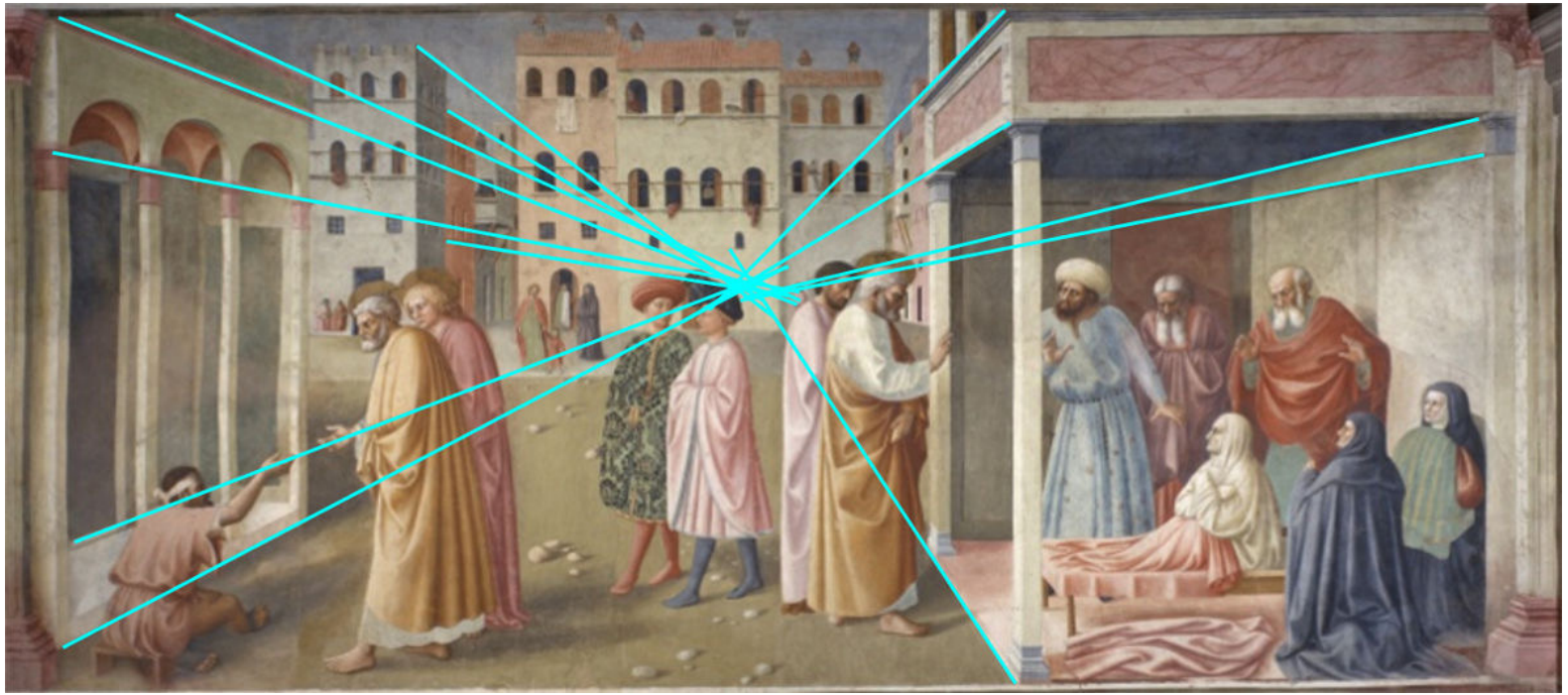
Parallel **lines** ‘converge’ @ **vanishing point**

Can lie both inside & outside the image



Most slides from Davide Scaramuzza

Perspective in Art



The Healing of the Cripple and Raising of Tabitha
Masolino, 1427

Projection: 3D → 2D coordinates

Length & area size
are not preserved!

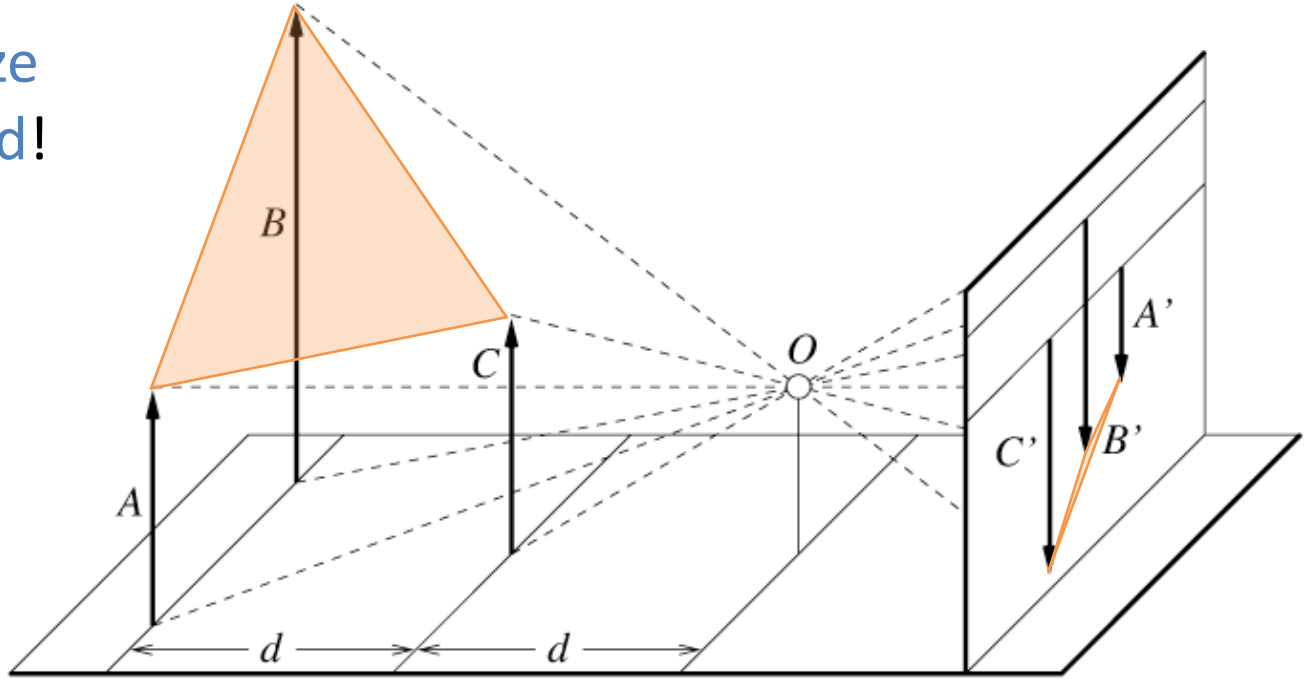
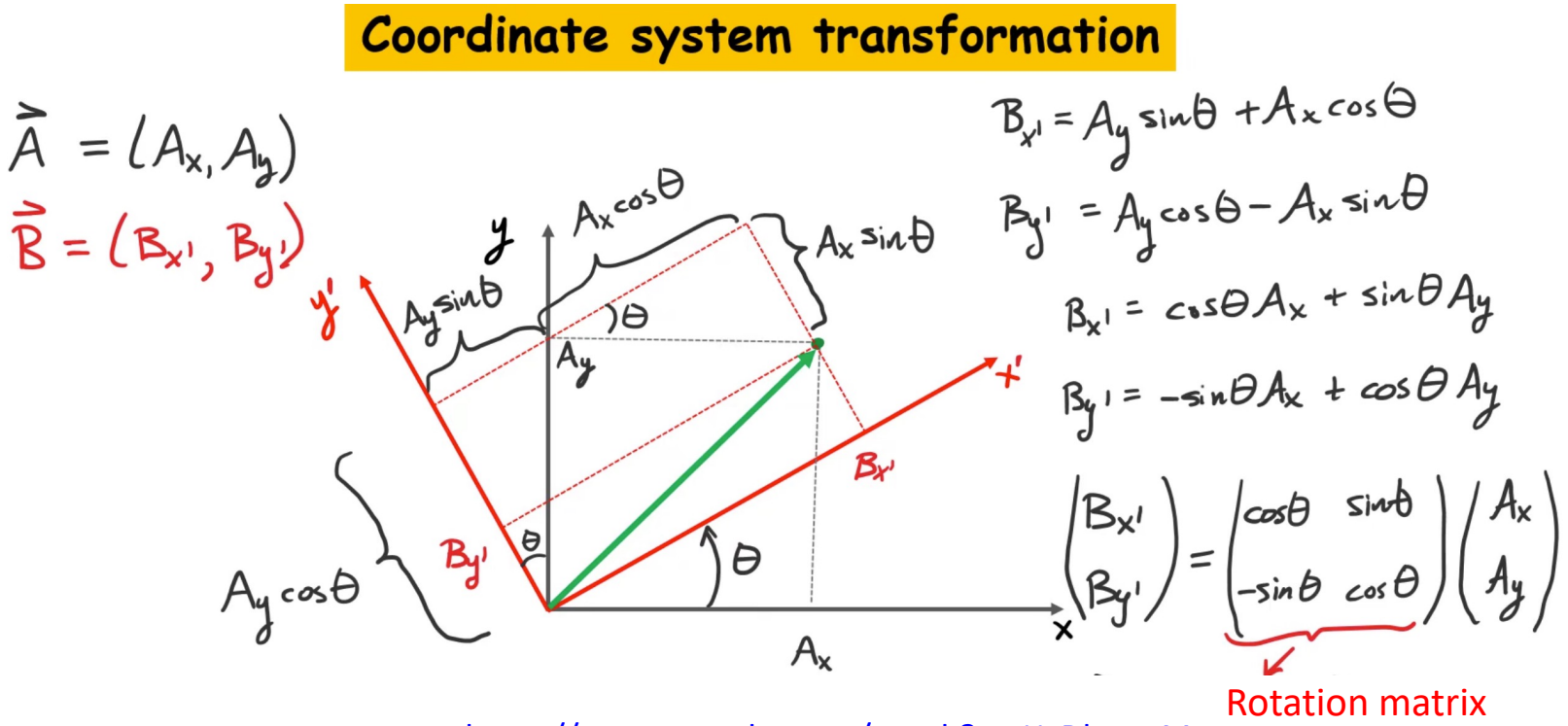


Figure by
David Forsyth

Homogeneous Coordinates

We can do rotation (but not translation) in Cartesian Coordinate by just doing matrix multiplication.



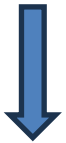
source -- https://www.youtube.com/watch?v=-HcDI_gyeMs

Homogeneous Coordinates

We use homogeneous coordinates to simplify the math for the translation operation.

Cartesian Coordinates

- **Rotation** → Applied as matrix mult.
- **Translation** → Cannot



We need a new tool
to simplify math

Homogeneous Coordinates

- **Rotation** → Applied as matrix mult.
- **Translation** → Applied as matrix mult.

Cartesian → Homog. coordinates

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

2D (image) coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

3D (scene) coordinates

Homog. → Cartesian coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

2D (image) coordinates

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow (x/w, y/w, z/w)$$

3D (scene) coordinates

Homogeneous Coordinates

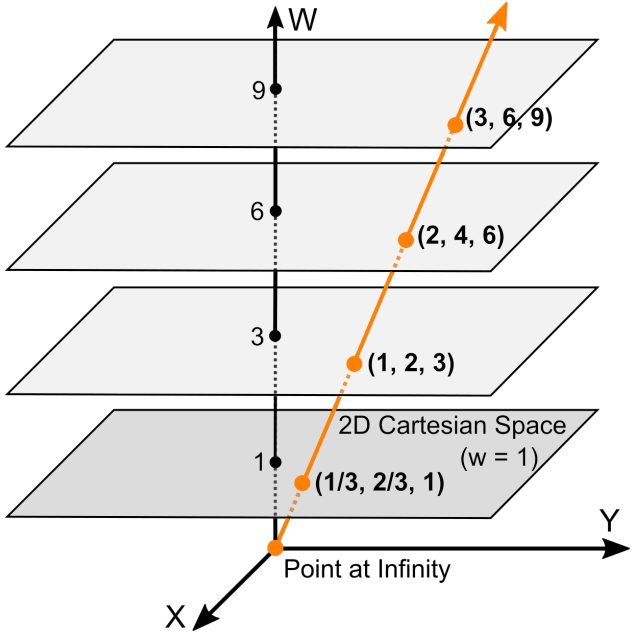
Invariant to scaling

$$k\begin{bmatrix} x \\ y \\ w \end{bmatrix} = \begin{bmatrix} kx \\ ky \\ kw \end{bmatrix} \Rightarrow \begin{bmatrix} \frac{kx}{kw} \\ \frac{ky}{kw} \end{bmatrix} = \begin{bmatrix} \frac{x}{w} \\ \frac{y}{w} \end{bmatrix}$$

$$k \neq 0$$

Homogeneous Coordinates

Cartesian Coordinates

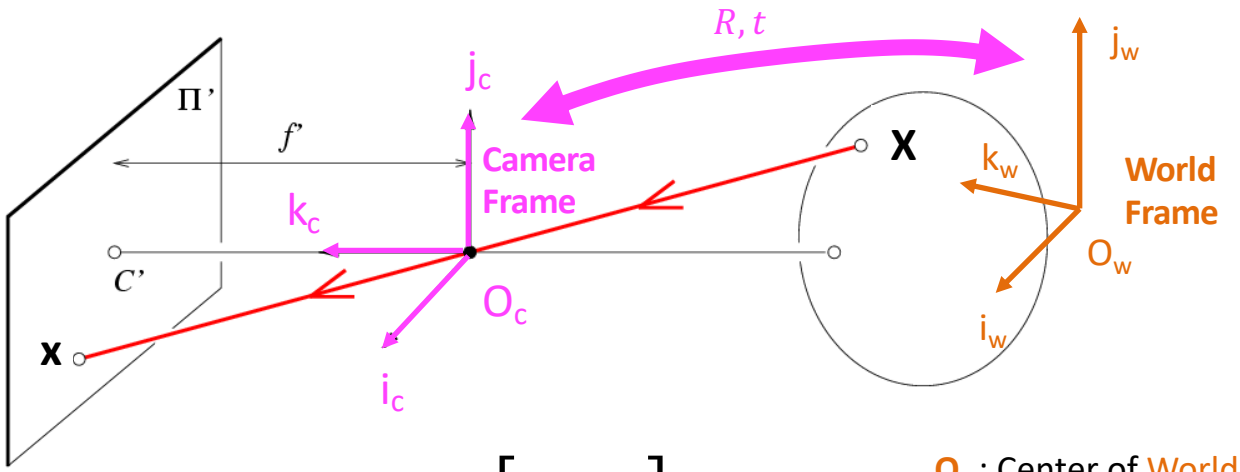


A **_Point_** in Cartesian Coordinates is …

a **___Ray___** in Homogeneous Coordinates

source -- https://www.songho.ca/math/homogeneous/homogeneous.html

3D→2D Projection Matrix



Camera Frame posed (R, t) w.r.t. World Frame

$$\mathbf{x} = \mathbf{K}\begin{bmatrix}\mathbf{R} & \mathbf{t}\end{bmatrix}\mathbf{X}$$

for now, no rotation and no translation: $\mathbf{R} = \mathbf{I}$ and $\mathbf{t} = \mathbf{0}$

$$\mathbf{x} = \mathbf{K}\begin{bmatrix}\mathbf{I} & \mathbf{0}\end{bmatrix}\mathbf{X}$$

Intrinsic Matrix

Extrinsic Matrix

O_w: Center of World Frame
O_c: Center of Camera Frame

X: 3D point in World Coordinates: (x,y,z,1)
x: 2D Image Coordinates: (u,v,1), up to scale w

K: Intrinsic Matrix (3x3) - internal parameters of a camera
R: Rotation (3x3)
t: Translation (3x1)

Note: different books use different notation!

Slide Credit: S. Saverese

3D→2D Projection Matrix

Extrinsic Assumptions

- **Camera Frame is World Frame**
- No rotation → $R = I$
- Camera at → $t = [0,0,0]$

Intrinsic Assumptions

- Pixels are perfect squares (f is the camera focal length) → $f_x = f_y = f$
- Sensor center aligns perfectly with the lens (no offset) → $u_0 = v_0 = 0$
- No skew (x and y axes are perfectly perpendicular) → $s = 0$

$$\mathbf{x} = \underbrace{\mathbf{K}[\mathbf{I} \quad \mathbf{0}]}_{\text{Projection Matrix}}\mathbf{X} \Rightarrow \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{K} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Cartesian Coord.
(after dividing with last element)

$$u = x\frac{f}{z}$$

$$v = y\frac{f}{z}$$

Slide Credit:
S. Saverese

3D→2D Projection Matrix

Extrinsic Assumptions

- **Camera Frame is World Frame**
- No rotation → $R = I$
- Camera at → $t = [0,0,0]$

Intrinsic Assumptions

- Pixels are perfect squares (f is the camera focal length) → $f_x = f_y = f$
- ~~Sensor center aligns perfectly with the lens (no offset) → $u_0 = v_0 = 0$~~
- No skew (x and y axes are perfectly perpendicular) → $s = 0$

$$\mathbf{x} = \underbrace{\mathbf{K}\begin{bmatrix}\mathbf{I} & \mathbf{0}\end{bmatrix}}_{\text{Projection Matrix}}\mathbf{X} \Rightarrow \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f & 0 & u_0 \\ 0 & f & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_{K} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Slide Credit:
S. Saverese

3D→2D Projection Matrix

Extrinsic Assumptions

- **Camera Frame is World Frame**
- No rotation → $R = I$
- Camera at → $t = [0,0,0]$

Intrinsic Assumptions

- ~~Pixels are perfect squares (f is the camera focal length)~~ → ~~$f_x = f_y = f$~~
- ~~Sensor center aligns perfectly with the lens (no offset)~~ → ~~$u_0 = v_0 = 0$~~
- No skew (x and y axes are perfectly perpendicular) → $s = 0$

$$\mathbf{x} = \underbrace{\mathbf{K}\begin{bmatrix}\mathbf{I} & \mathbf{0}\end{bmatrix}}_{\text{Projection Matrix}}\mathbf{X} \Rightarrow \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & 0 & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_{K} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Slide Credit:
S. Saverese

3D→2D Projection Matrix

Extrinsic Assumptions

- **Camera Frame is World Frame**
- No rotation → $R = I$
- Camera at → $t = [0,0,0]$

Intrinsic Assumptions

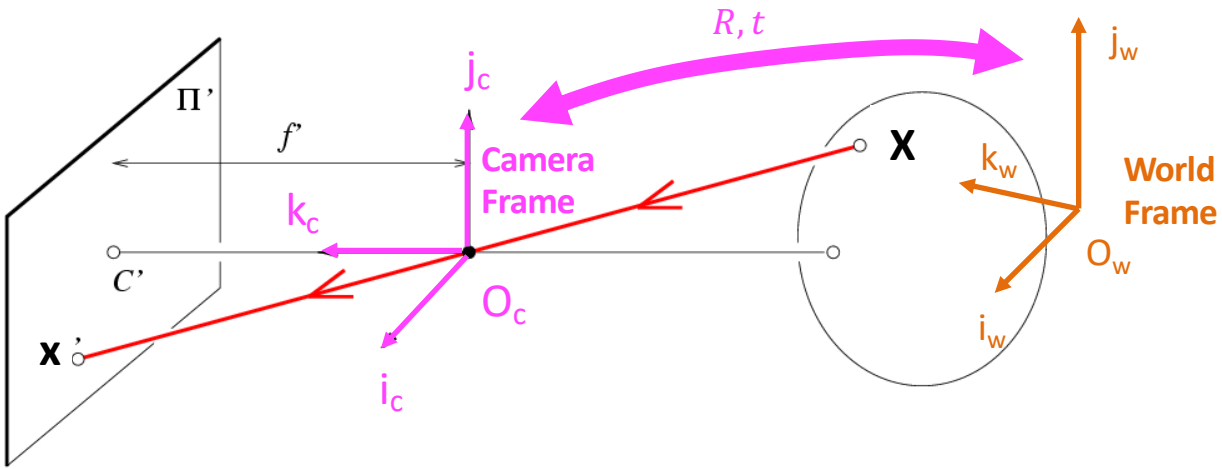
- ~~Pixels are perfect squares (f is the camera focal length)~~ → ~~$f_x = f_y = f$~~
- ~~Sensor center aligns perfectly with the lens (no offset)~~ → ~~$u_0 = v_0 = 0$~~
- ~~No skew (x and y axes are perfectly perpendicular)~~ → ~~$s = 0$~~

The most general form of Intrinsics K

$$\mathbf{x} = \underbrace{\mathbf{K}[\mathbf{I} \quad \mathbf{0}]}_{\text{Projection Matrix}}\mathbf{X} \Rightarrow \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \underbrace{\begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix}}_{K} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Slide Credit:
S. Saverese

3D Rotation of Points



We have explained

$$\mathbf{x} = \mathbf{K}\begin{bmatrix}\mathbf{I} & \mathbf{0}\end{bmatrix}\mathbf{X}$$

Now, let us consider rotation and translation

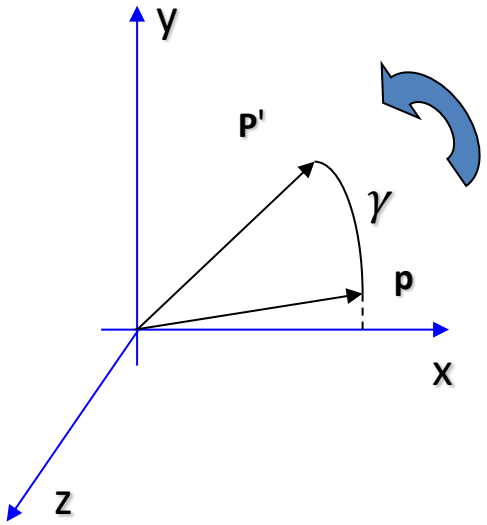
$$\mathbf{x} = \mathbf{K}\begin{bmatrix}\mathbf{R} & \mathbf{t}\end{bmatrix}\mathbf{X}$$

Intrinsic Matrix

Extrinsic Matrix

Think about this: Why not just always assume that the world frame is the camera frame?

3D Rotation of Points



Rotation around
coordinate axes
(counter-clockwise)

α → Angle around axis-x
β → Angle around axis-y
γ → Angle around axis-z

$$R_x(\alpha) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\alpha & -\sin\alpha \\ 0 & \sin\alpha & \cos\alpha \end{bmatrix}$$

$$R_y(\beta) = \begin{bmatrix} \cos\beta & 0 & \sin\beta \\ 0 & 1 & 0 \\ -\sin\beta & 0 & \cos\beta \end{bmatrix}$$

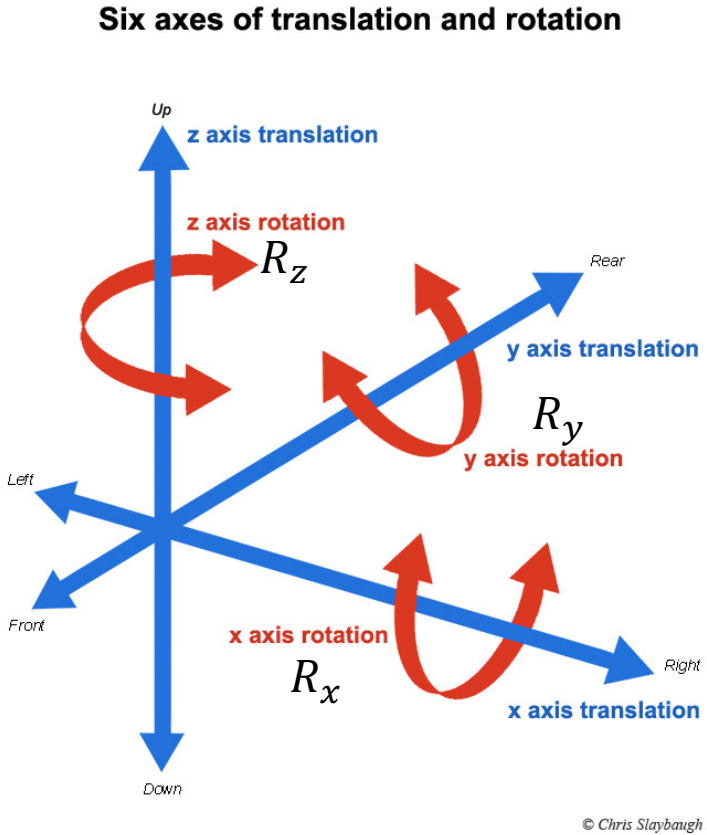
$$R_z(\gamma) = \begin{bmatrix} \cos\gamma & -\sin\gamma & 0 \\ \sin\gamma & \cos\gamma & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Each of R_x, R_y, R_z has 1 DoF
(number of free parameters)

Complex Rotations → Multiply R_x & R_y & R_z
(the order matters)

Slide Credit:
S. Saverese

3D Rotation of Points



source -- https://photomacrography.net/forum/viewtopic.php?p=134654

Intrinsics & Extrinsics

Camera Frame
Rotated & Translated
w.r.t. World Frame

$$\mathbf{x} = \mathbf{K}\begin{bmatrix}\mathbf{R} & \mathbf{t}\end{bmatrix}\mathbf{X}$$

$$w\begin{bmatrix}u\\v\\1\end{bmatrix} = \begin{bmatrix}f_x & s & u_0\\0 & f_y & v_0\\0 & 0 & 1\end{bmatrix}\begin{bmatrix}r_{11} & r_{12} & r_{13} & t_x\\r_{21} & r_{22} & r_{23} & t_y\\r_{31} & r_{32} & r_{33} & t_z\end{bmatrix}\begin{bmatrix}x\\y\\z\\1\end{bmatrix}$$

3 + 3

Degrees of Freedom (DoF) → 5 + 6 = 11 DoFs in total

(how many free parameters)

(Why not 12 for the extrinsic matrix?
Hint: think about the rotation matrix)

Intrinsics & Extrinsics

What if
no rotation?

$$\mathbf{x} = \mathbf{K}\begin{bmatrix}\mathbf{R} & \mathbf{t}\end{bmatrix}\mathbf{X}$$



$$w\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & s & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \end{bmatrix}\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

Intrinsics & Extrinsics

Identity Rotation →
$R = I$

$$\mathbf{x} = \mathbf{K}\begin{bmatrix}\mathbf{I} & \mathbf{t}\end{bmatrix}\mathbf{X}$$

$$w\begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & u_0 \\ 0 & f_y & v_0 \\ 0 & 0 & 1 \end{bmatrix}\begin{bmatrix} 1 & 0 & 0 & t_x \\ 0 & 1 & 0 & t_y \\ 0 & 0 & 1 & t_z \end{bmatrix}\begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

What if
no rotation &
no translation?

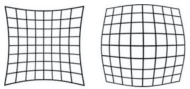
→ The World & Camera Frames coincide →

Only the Intrinsics suffice for projection

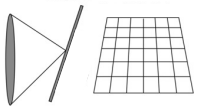
Intrinsics – Distorted Image

Front
‘fisheye’ camera
of FrodoBot
(IRLab, UvA)





Radial distortion



Tangential distortion

Camera is uncalibrated ☹

Need to: Calibrate camera → estimate intrinsics

Undistort image via estimated intrinsics

Intrinsics – Distorted Image

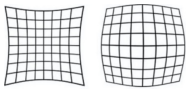


OpenCV tutorial on **Camera Calibration** & image **Undistortion**

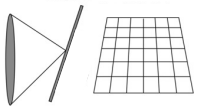
https://docs.opencv.org/4.x/dc/dbb/tutorial_py_calibration.html

Check the "Brown-Conrady model" if you are interested in the distortion math.

$$Distortion\ coefs = (k_1 \quad k_2 \quad p_1 \quad p_2 \quad k_3)$$



Radial distortion → $x_{distorted} = x(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$ $y_{distorted} = y(1 + k_1 r^2 + k_2 r^4 + k_3 r^6)$



Tangential distortion → $x_{distorted} = x + [2p_1 xy + p_2(r^2 + 2x^2)]$ $y_{distorted} = y + [p_1(r^2 + 2y^2) + 2p_2 xy]$

Intrinsics – Undistorted Image – 0 Elements



```
dist_coeff = [-0.000, 0.0000, 0.00000, -0.00000, -0.00000]
```

Intrinsics – Undistorted Image – 1 Element



```
dist_coeff = [-0.217, 0.0000, 0.00000, -0.00000, -0.00000]
```

Intrinsics – Undistorted Image – 2 Elements



```
dist_coeff = [-0.217, 0.0537, 0.00000, -0.00000, -0.00000]
```

Intrinsics – Undistorted Image – 3 Elements



```
dist_coeff = [-0.217, 0.0537, 0.00185, -0.00000, -0.00000]
```

Intrinsics – Undistorted Image – 4 Elements



```
dist_coeff = [-0.217, 0.0537, 0.00185, -0.00210, -0.00000]
```

Intrinsics – Undistorted Image – 5 Elements



```
dist_coeff = [-0.217, 0.0537, 0.00185, -0.00210, -0.00600]
```

Intrinsics – Undistorted Image



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

```
h,w = [1024, 576]
```

Intrinsics – Undistorted Image



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Intrinsics – ~~Un~~distorted Image



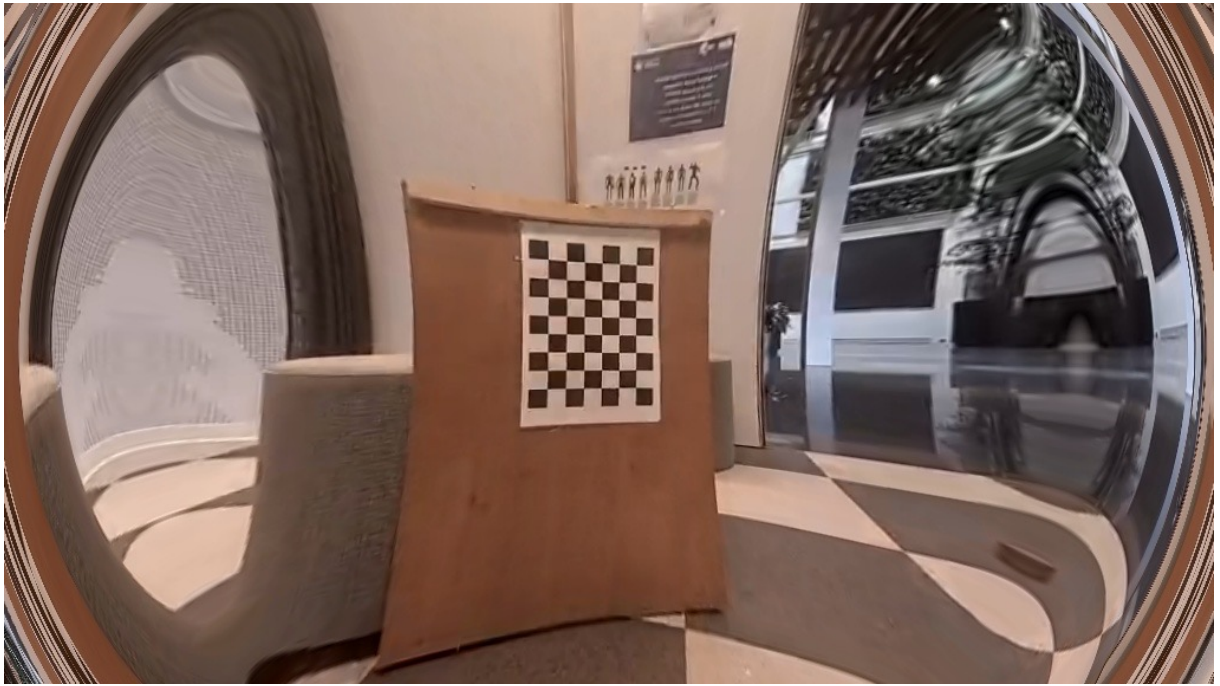
$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

Mess up: f_x, f_y, c_x, c_y = [407.860, 407.866, 512.000, 288.000]

Original: f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Intrinsics – ~~Un~~distorted Image



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

Mess up: $f_x, f_y, c_x, c_y = [200.000, 200.000, 512.000, 288.000]$

Original: $f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]$

Intrinsics – ~~Un~~distorted Image



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

Mess up: $f_x, f_y, c_x, c_y = [300.000, 300.000, 512.000, 288.000]$

Original: $f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]$

Intrinsics – ~~Un~~distorted Image



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

Mess up: f_x, f_y, c_x, c_y = [400.000, 400.000, 512.000, 288.000]

Original: f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Intrinsics – ~~Un~~distorted Image



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

Mess up: $f_x, f_y, c_x, c_y = [600.000, 600.000, 512.000, 288.000]$

Original: $f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]$

Intrinsics – ~~Un~~distorted Image



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

Mess up: f_x, f_y, c_x, c_y = [800.000, 800.000, 512.000, 288.000]

Original: f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Intrinsics – Undistorted Image - Skew

Skew

s = [000.0]



$$camera\ matrix = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

h,w = [1024, 576]

f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Intrinsics – ~~Un~~distorted Image - Skew

Mess up:

Skew

`s = [100.0]`



$$camera\ matrix = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

`h,w = [1024, 576]`

f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Intrinsics – ~~Un~~distorted Image - Skew

Mess up:

Skew

`s = [200.0]`



$$camera\ matrix = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

`h,w = [1024, 576]`

f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Intrinsics – ~~Un~~distorted Image - Skew

Mess up:

Skew

`s = [400.0]`



$$camera\ matrix = \begin{bmatrix} f_x & s & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

`h,w = [1024, 576]`

f_x, f_y, c_x, c_y = [407.860, 407.866, 533.301, 278.699]

Outline

- Pinhole Camera
- Geometric Image Formation (where an object appears)
- Photometric Image Formation (how bright and what color it is)
- Image Representation
- Color

Image Formation

Four main factors influence image intensity values

- Illumination of scene
- Geometry of scene
- Reflectance of visible surfaces
- Camera view & optics

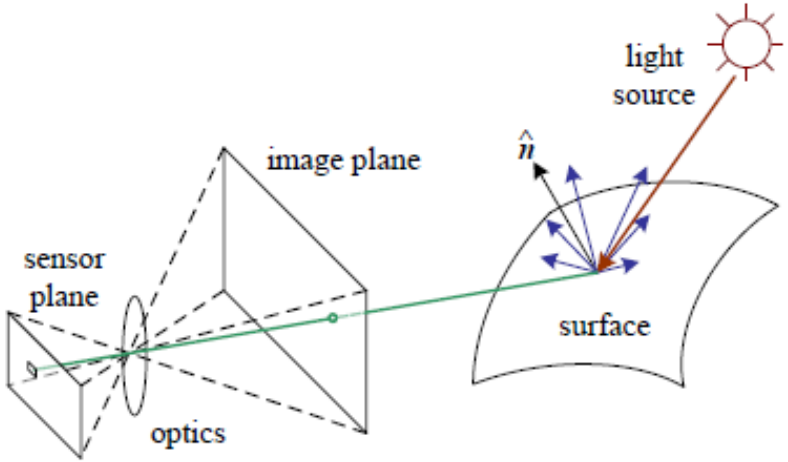


Image Formation – Photometry

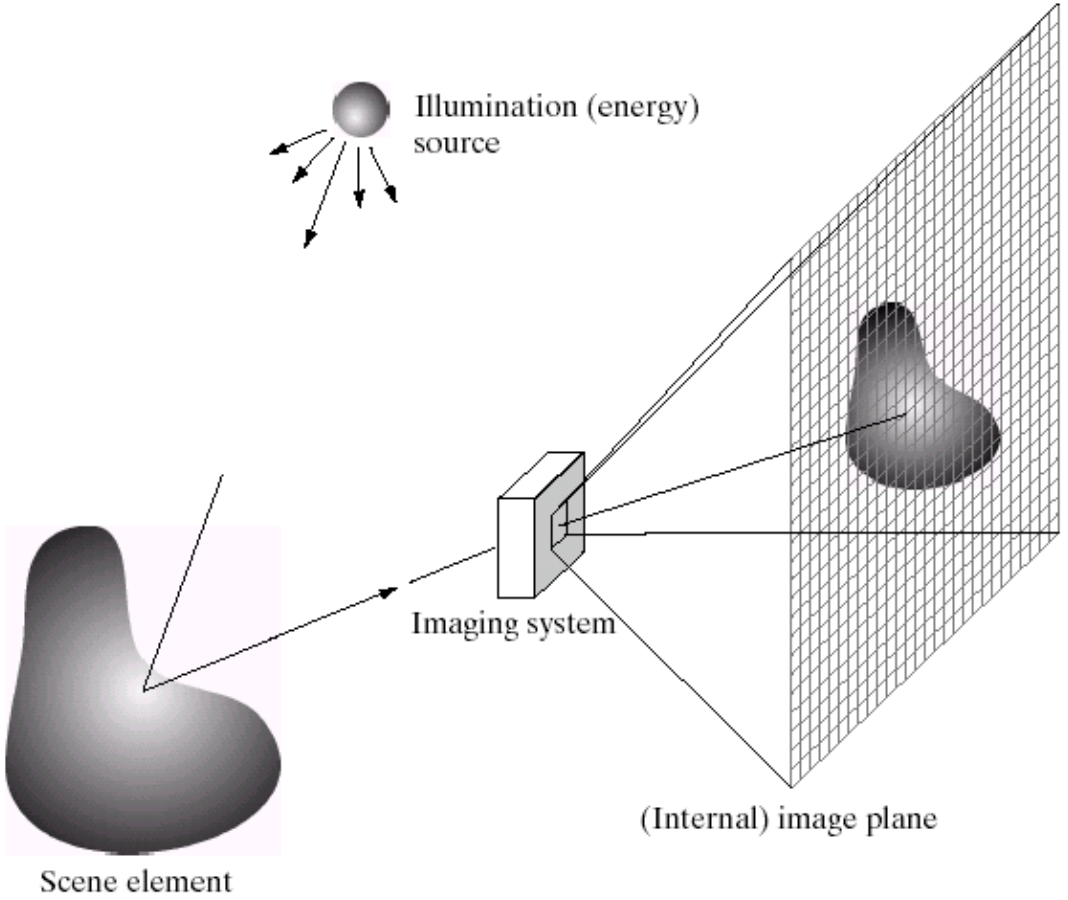
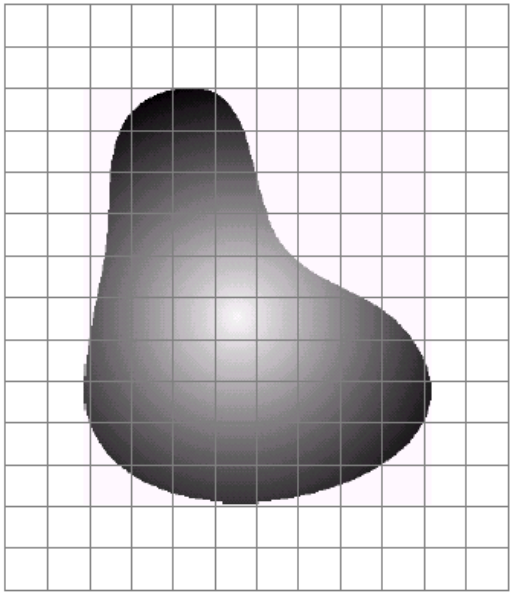
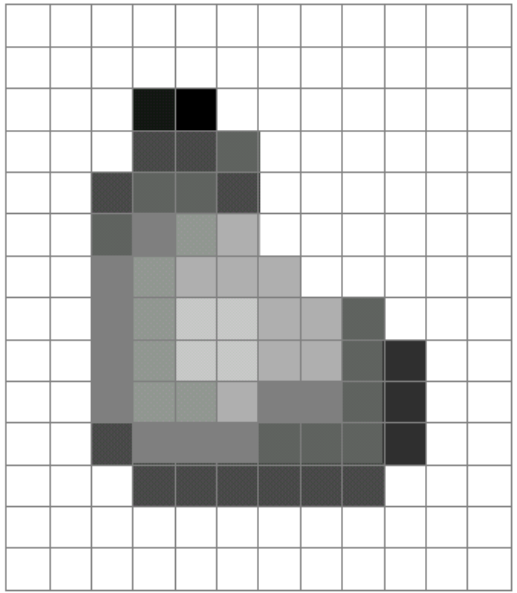


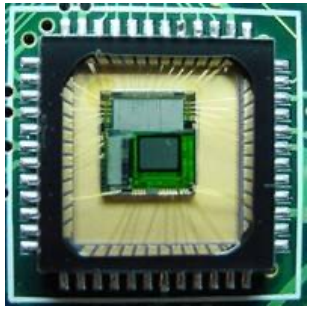
Image Formation – Sensor Array



Continuous image projected onto a sensor array

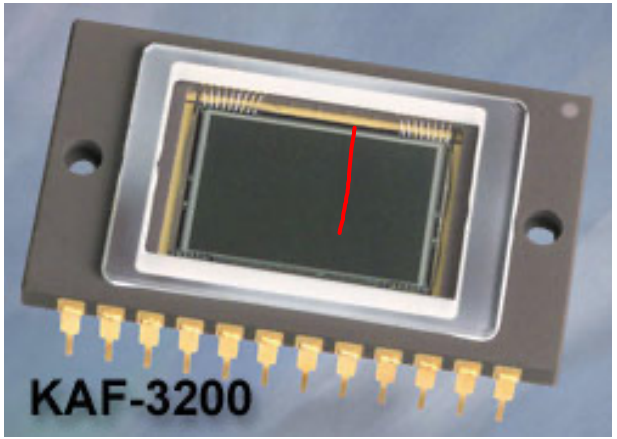


Result of image
sampling & → Spatial position
quantization → Numerical value



CCD/CMOS sensor

Image Formation – Sensor Array



CCD KAF-3200E from Kodak
(2184 x 1472 pixels)
(Pixel size 6.8 microns^2)

Charge-Coupled Device (CCD)

- Converts continuous image → digital image
- Contains an array of light-sensor units
- Converts photons → electric charges accumulated in each sensor unit

Image Formation – Sensor – Imperfections



Low light



Dynamic Range



CCD overflow

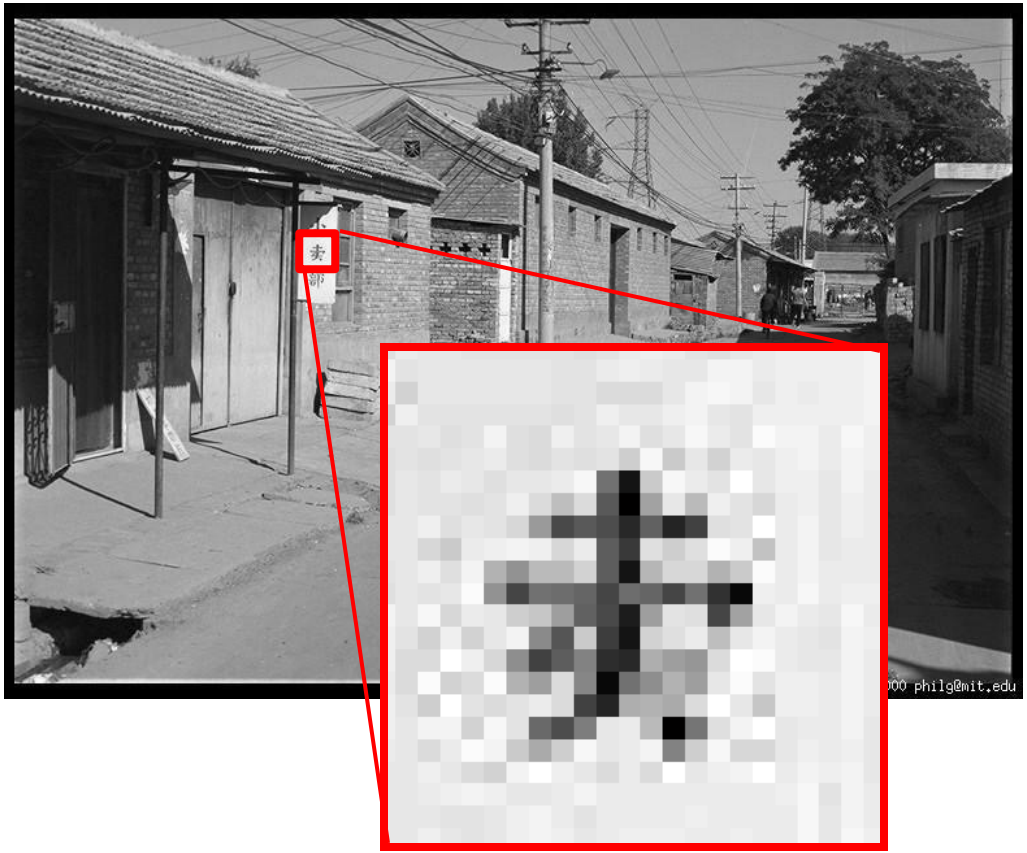


Rolling Shutter

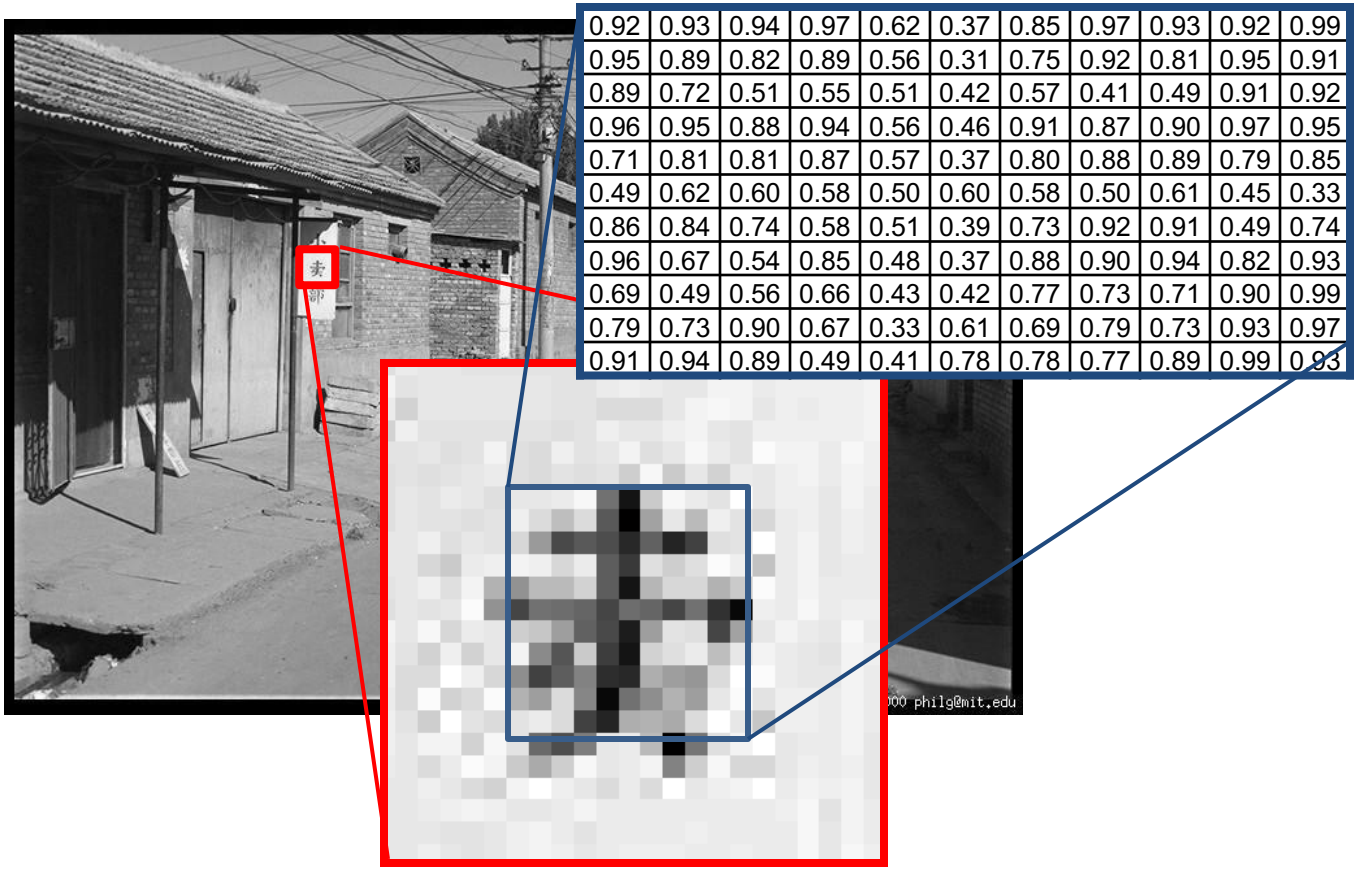
Outline

- Pinhole Camera
- Geometric Image Formation (where an object appears)
- Photometric Image Formation (how bright and what color it is)
- Image Representation
- Color

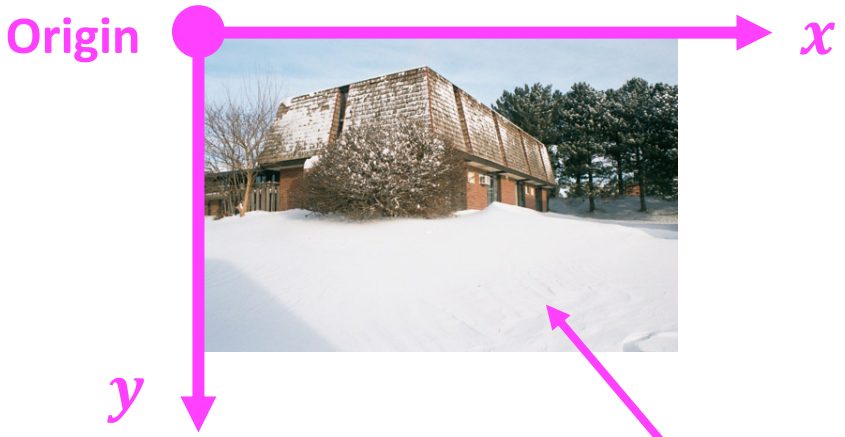
Digital Image – Pixel Matrix



Digital Image – Pixel Matrix

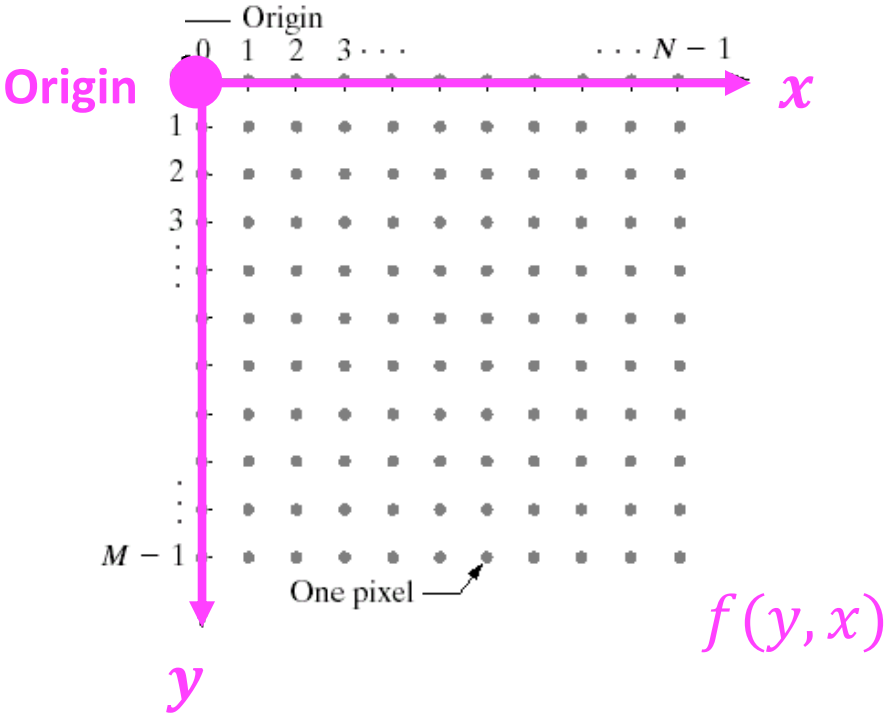


Digital image – Fundamentals



- **Image is a function** of spatial coordinates
- **Spatial coordinates**:
  - (x, y) for 2D case – e.g. photograph
  - (x, y, t) for video
  - (x, y, z) for 3D case – e.g. CT scans
- The function f may represent intensity (for greyscale images) or color (for color images) or other associated values

Digital image – Coordinate System



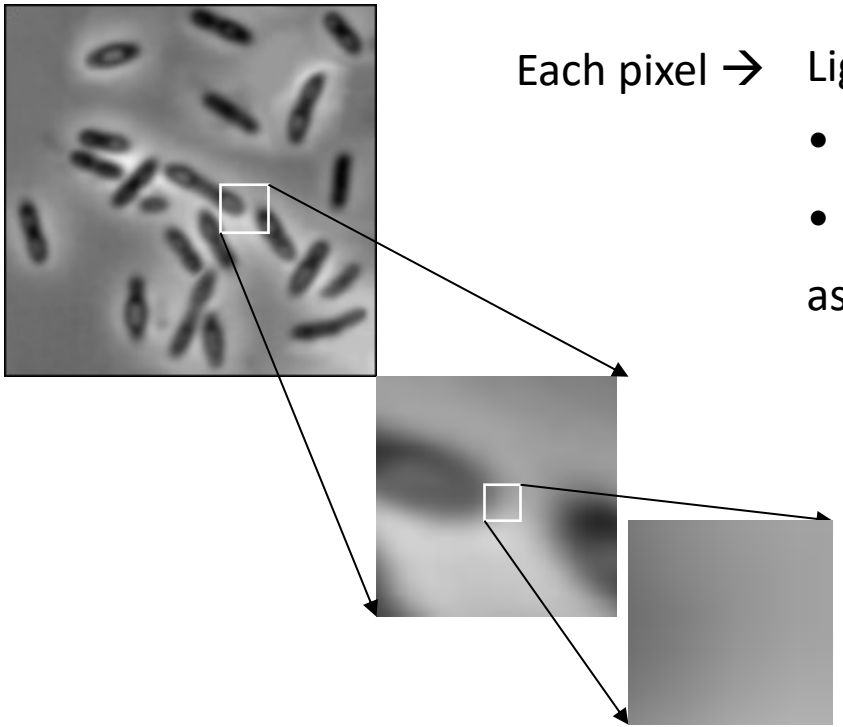
Regular grid of sampling points

Sampling point called a **Pixel**
(PICTure ELement)

Images from
Rafael C. Gonzalez and Richard E. Wood,
Digital Image Processing, 2nd Edition

Digital Image – Grayscale

Intensity- or **Grayscale-** image



Each pixel → Light intensity in the range of

- [0.0, …, 1.0] (`float`)
- [0, …, 255] (`uint8`)

as [black, …, white]

Digital Image – Color

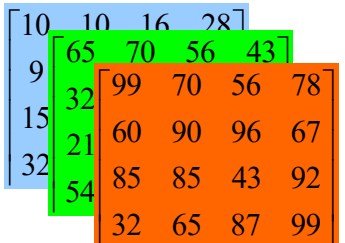
Color (RGB) image



Each pixel → Contains a 3-element vector:

- [0, …, 255] → Red
- [0, …, 255] → Green
- [0, …, 255] → Blue

RGB components



Outline

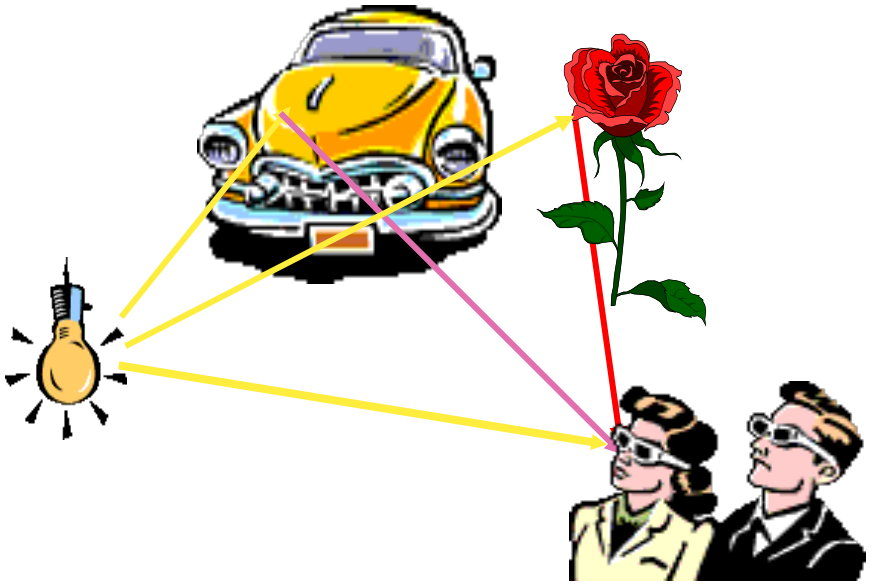
- Pinhole Camera
- Geometric Image Formation (where an object appears)
- Photometric Image Formation (how bright and what color it is)
- Image Representation
- Color

What makes for an image?

Light source

Object (s)

Observer / Sensor

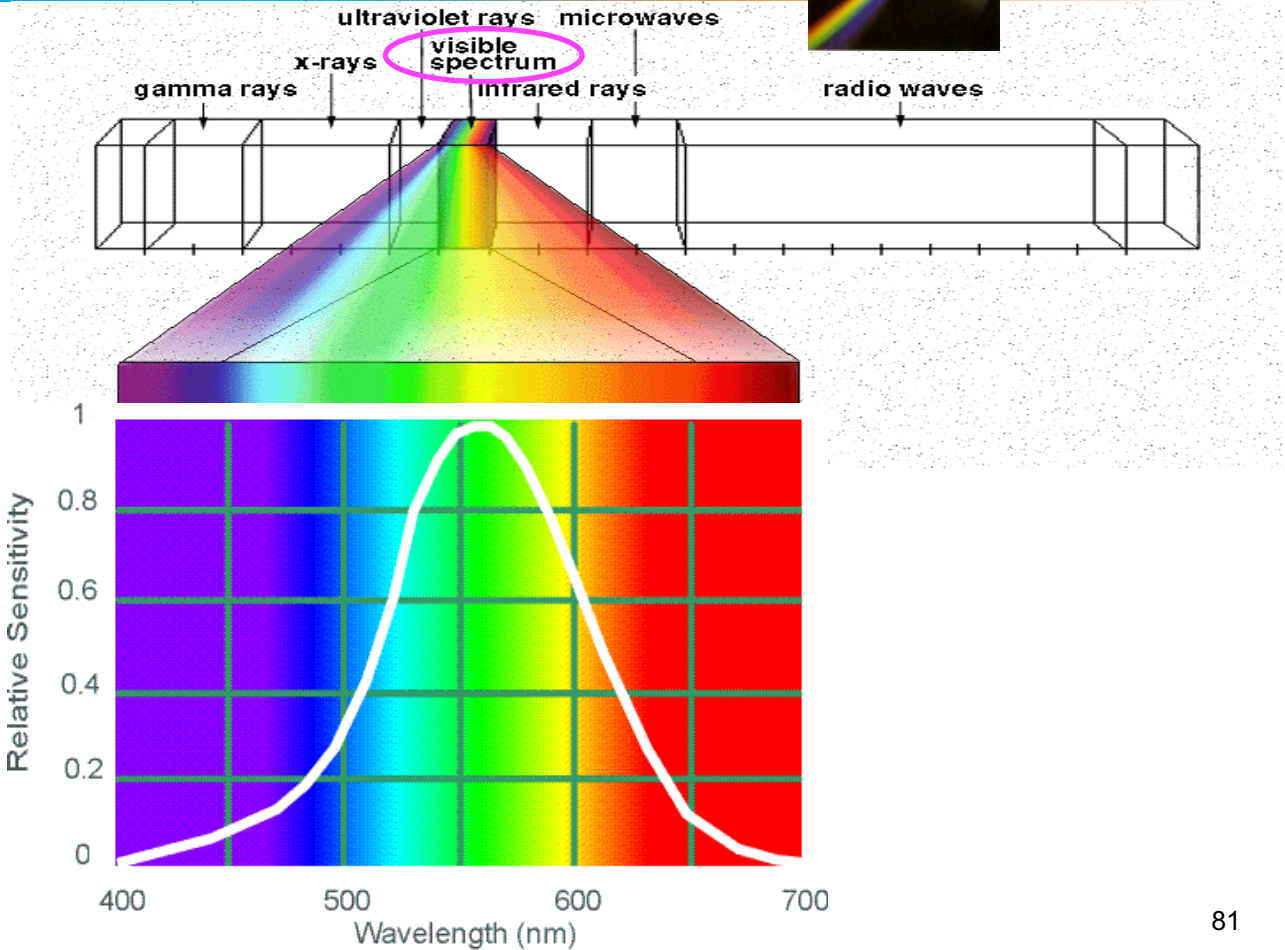


Humans – Sensitivity Function



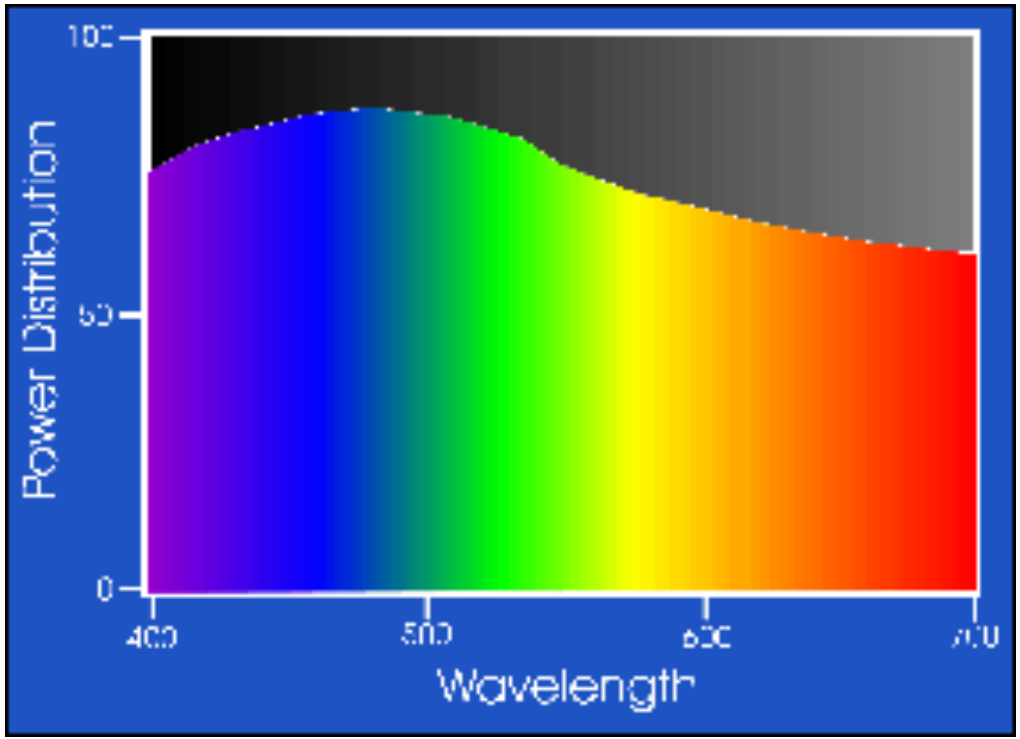
Most sensitive at middle wavelengths

Sensitivity falls off for long & short wavelengths

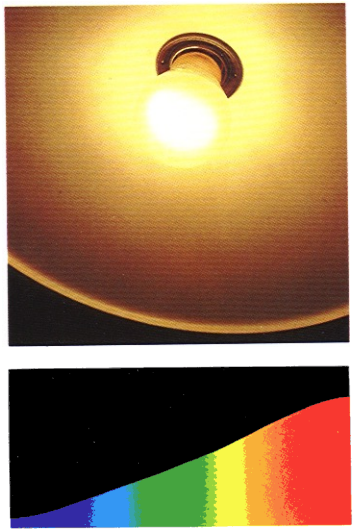


http://www.yorku.ca/eye/photopik.htm

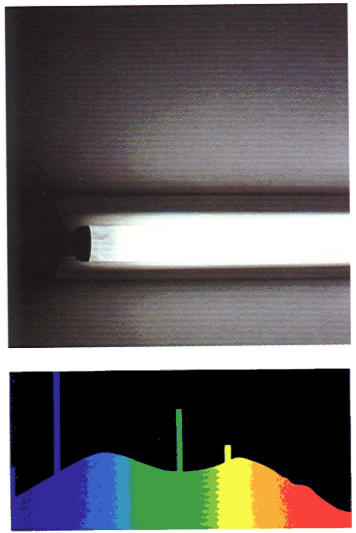
Light Sources – Spectral Power Distribution



Light Sources – Types

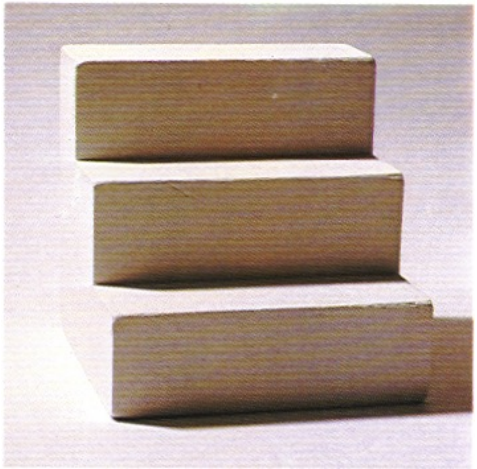


Incandescent lamp

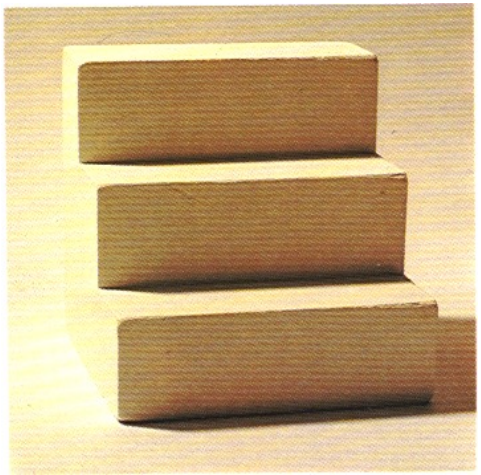


Fluorescent lamp

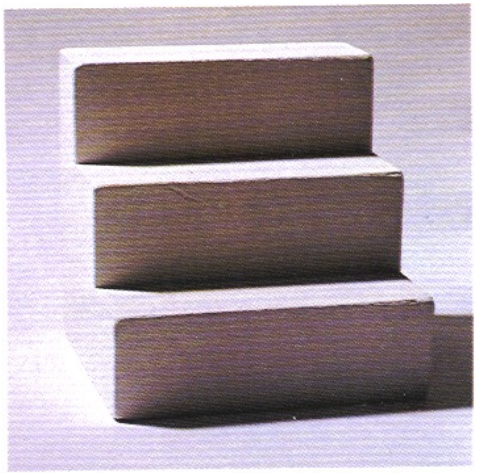
Light Sources – Influence



Average daylight



Incandescent lamp



Fluorescent lamp

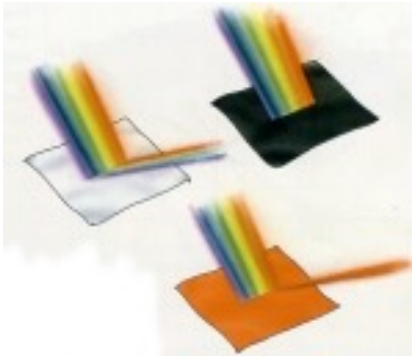
Object Colors

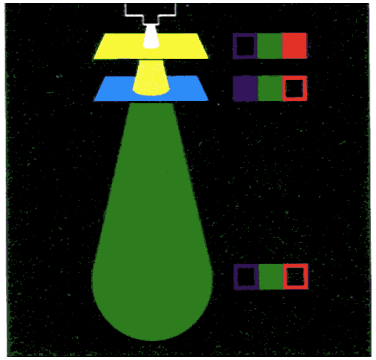
Materials:

- Transparent
- Opaque

Spectral Reflectance $\rho(\lambda)$

Wavelength of light

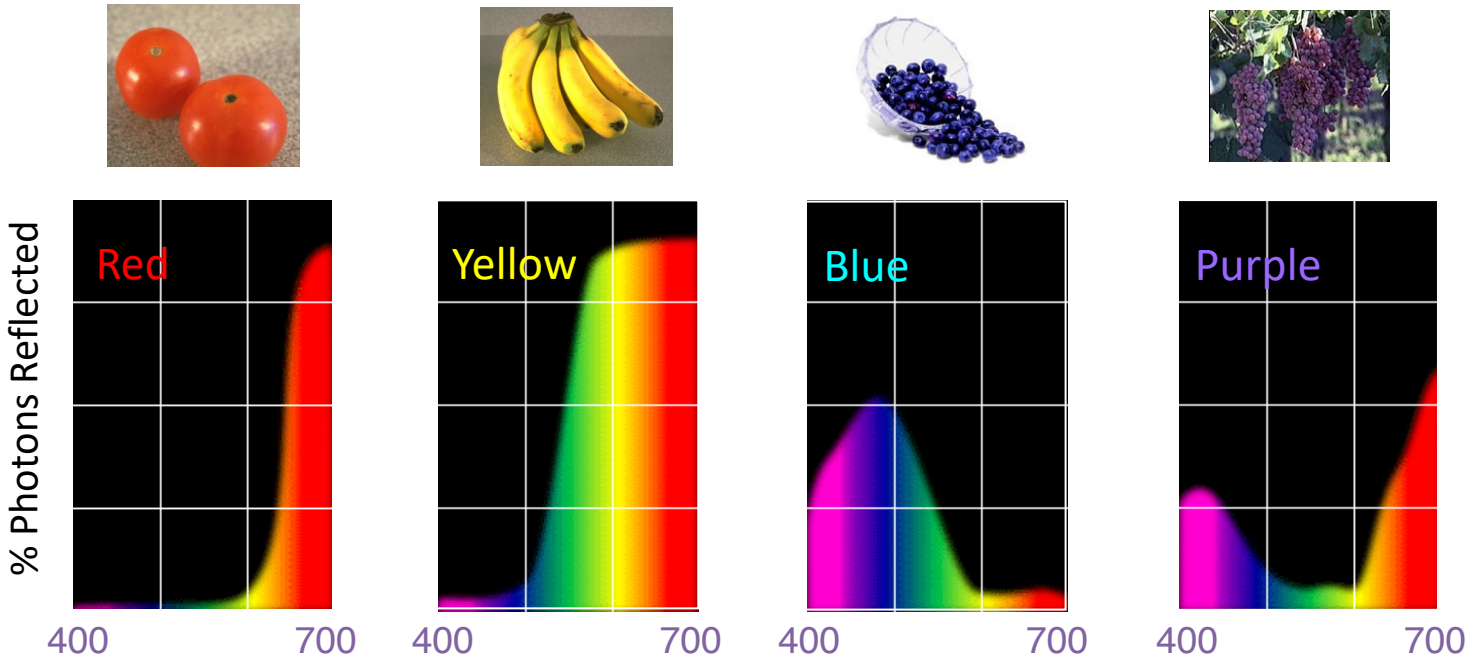






Object Colors

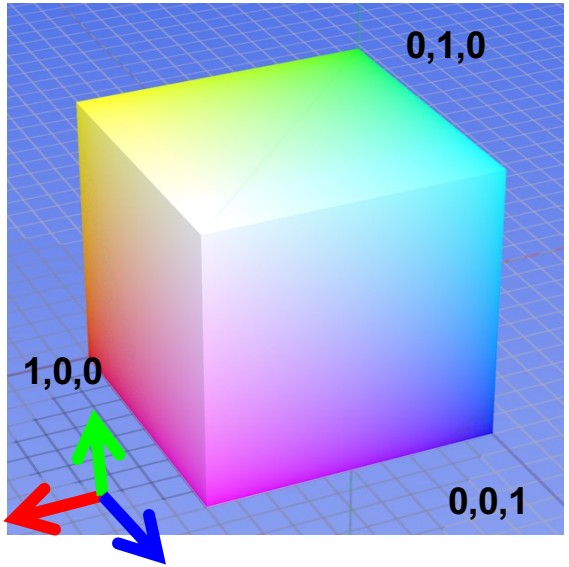
Examples of reflectance spectra of surfaces



© Stephen E. Palmer, 2002

RGB space





Default color space

Drawbacks:

- Strongly correlated channels
- Perceptually non-meaningful



R
(G=0,B=0)



G
(R=0,B=0)



B
(R=0,G=0)



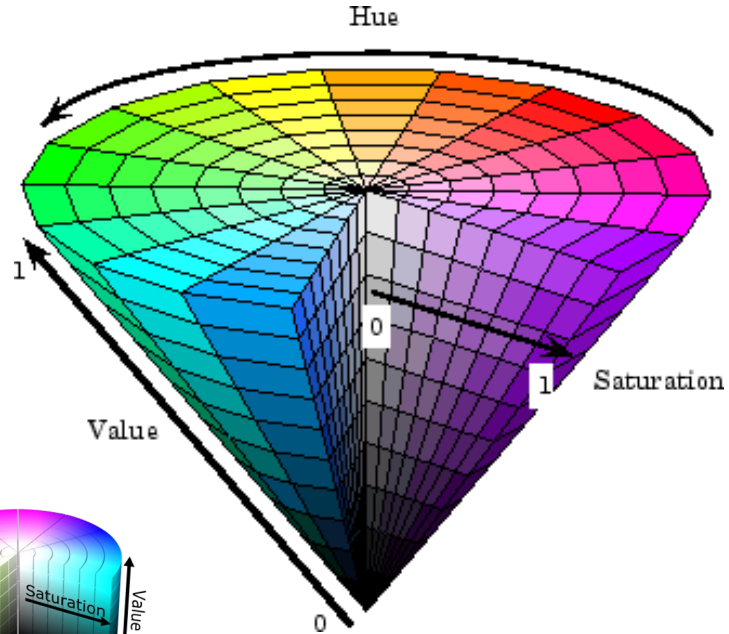
OpenCV order is

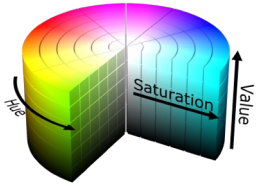
BGR

Image from: http://en.wikipedia.org/wiki/File:RGB_color_solid_cube.png

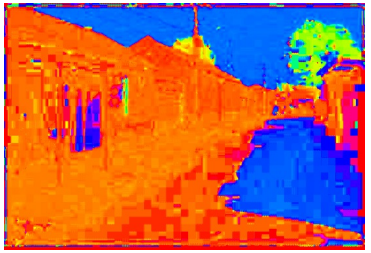
HSV Color Space







Intuitive color space



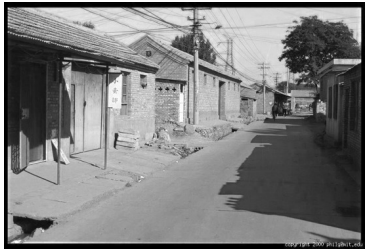
H
(S=1,V=1)

Hue



S
(H=1,V=1)

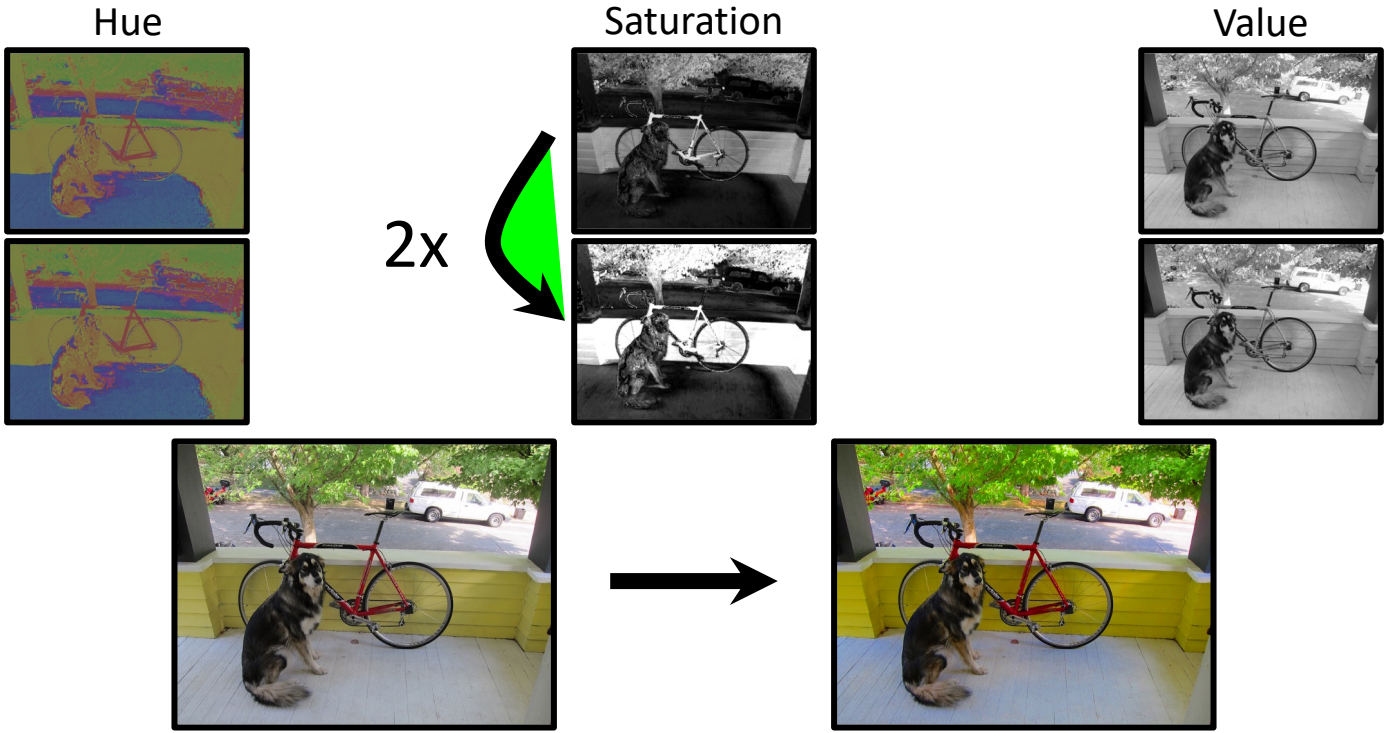
Saturation



V
(H=1,S=0)

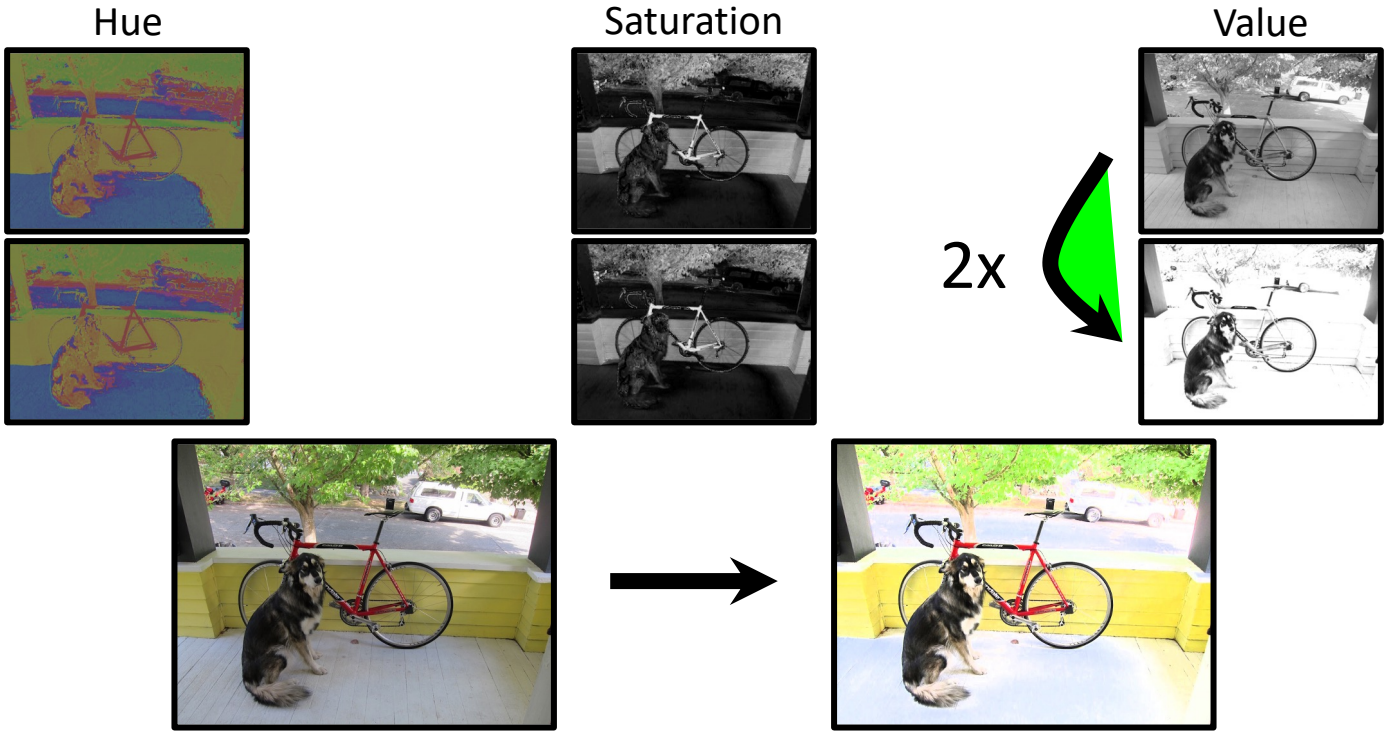
Value

HSV Color Space



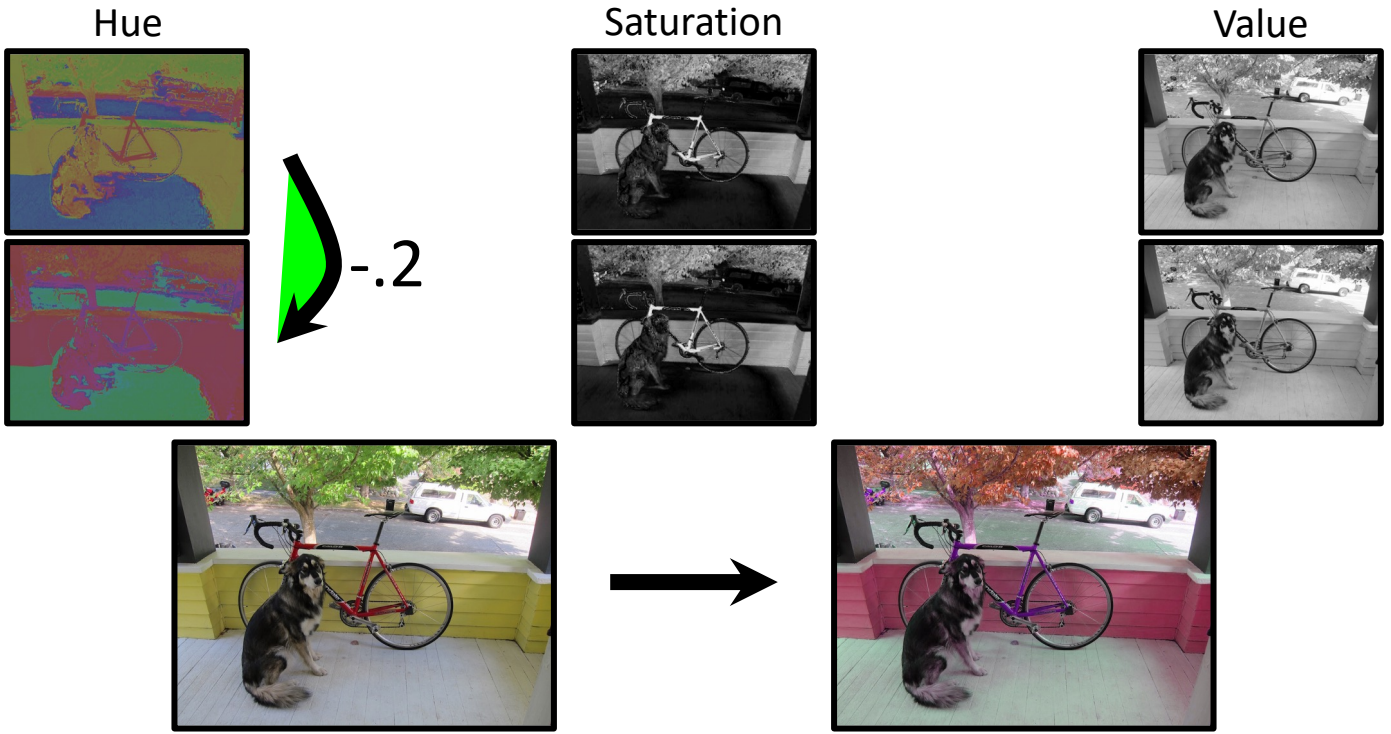
Higher Saturation → More intense colors

HSV Color Space



Higher Value → Brighter colors

HSV Color Space



Change Hue → Change Colors

What does this mean for computers?

- We represent images as grids of pixels
  - Each pixel has a 3-component color: RGB
- Can represent color with 3 numbers
  - #ff00ff; (1.0, 0.0, 1.0); (255, 0, 255); etc…
- Not every color can be represented in RGB (there are many other color spaces, such as CMYK, CIE, …)
- RGB is made to ‘trick’ humans – not to be accurate

Disclaimer

Many of the slides used here are obtained from online resources (including many open lecture materials) without appropriate acknowledgement. They are used here for the sole purpose of classroom teaching. All the credit and all the copyrights belong to the original authors. You should not copy it, redistribute it, put it online, or use it for any other purposes than for this course.